CM06 - Pointers

Pointer definition, dereference, pointer-to-pointer, arithmetic

Louis Ledoux

2026-07-05

Table of contents

1 Pointer Type and Use

Live pacing: about 8 min

Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

1.1 Main Path

- Start from the question: What does a pointer value name?
- Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- Keep only this slide on the horizontal path during the live lecture.

1.2 Worked Example

- Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- Representative source slide: 200
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

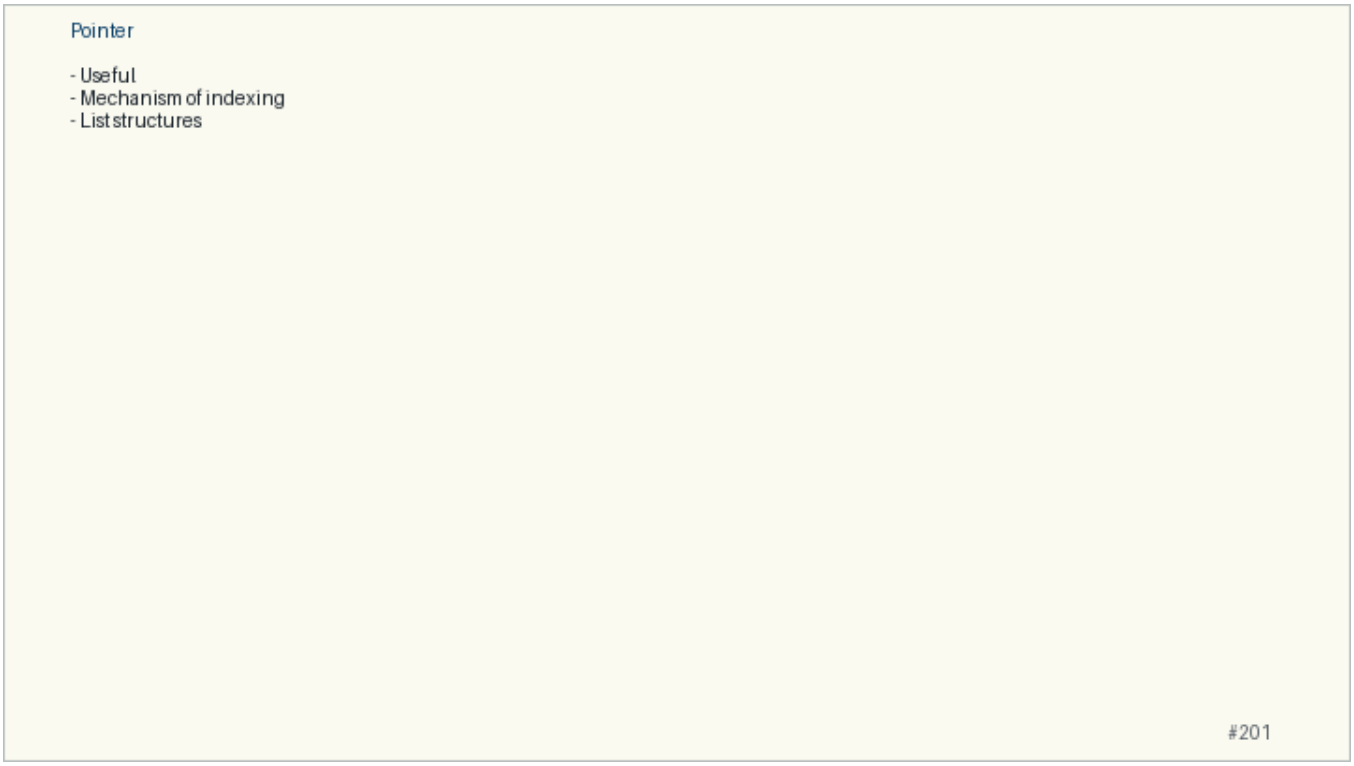
1.3 Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

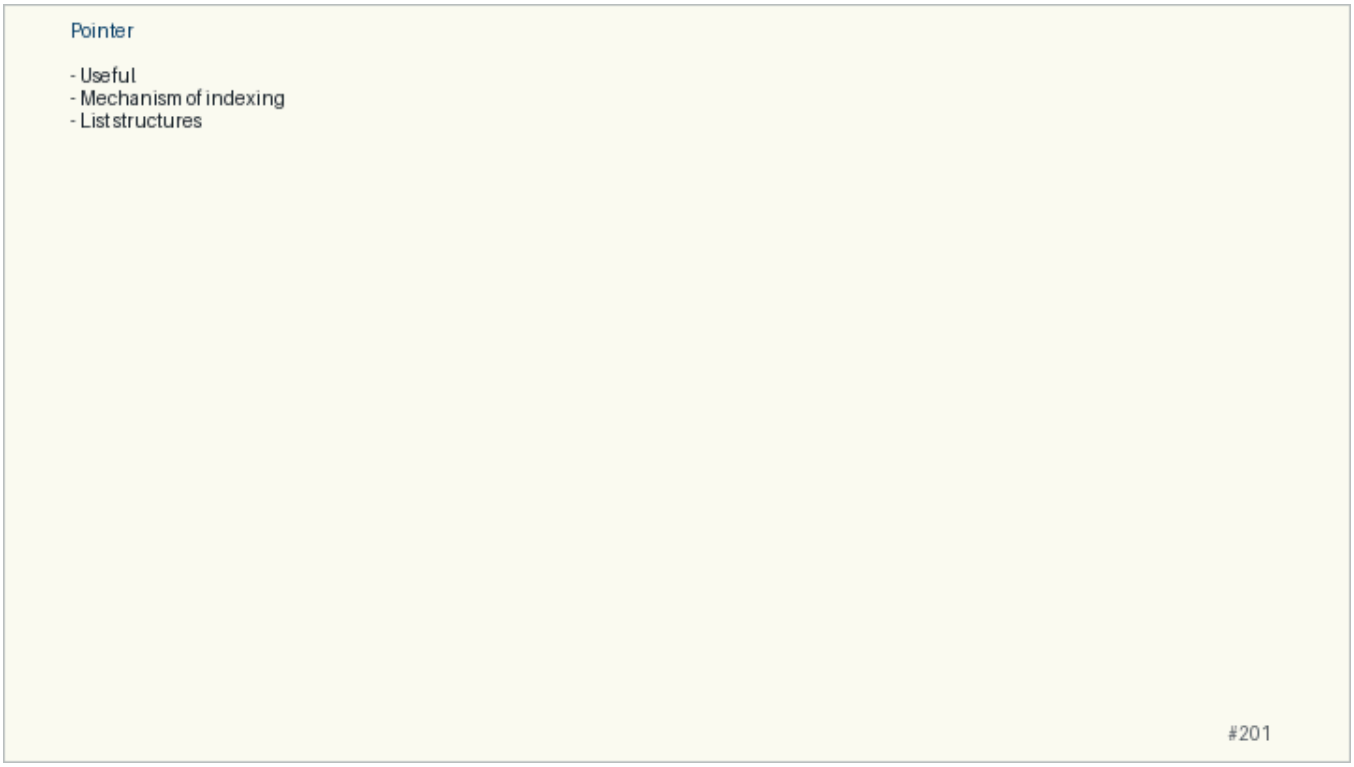


1.5 Backup Index

Original slides 199-201

slide 199

slide 200



slide 201

Anchors: CM 6-7 Pointers and Arrays; Pointer type; Pointer

1.6 Backup: CM 6-7 Pointers and Arrays

Original slide 199

1.7 Backup: Pointer type

Original slide 200

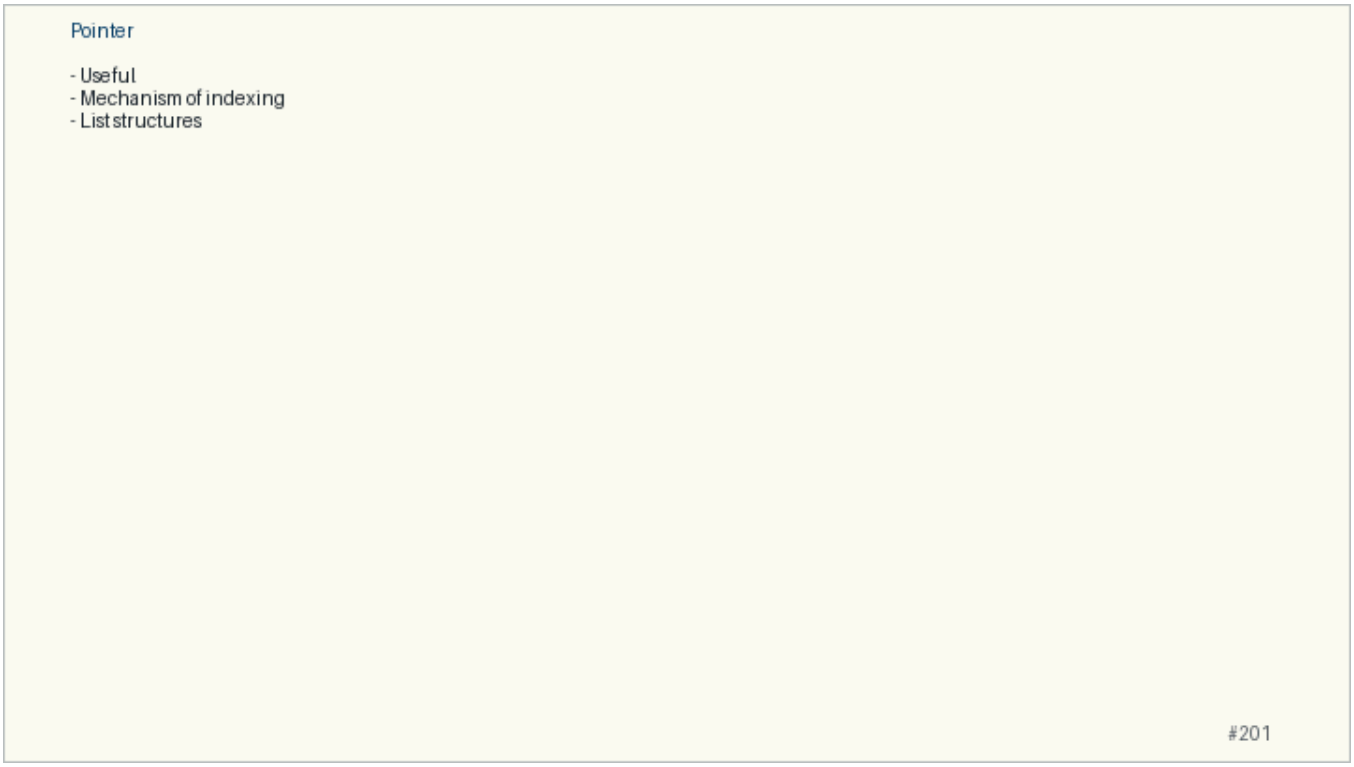
- Syntax:
- type * identifier;
- Function:

- The pointer type, *, is applied over a specific type of an object
- The type can be anything: integer, character, float, pointer, structure, function etc.
- The identifier points (refers) to an objet of that type
- It contains the address of the object that it is pointing to

1.8 Backup: Pointer

Original slide 201

- Useful
- Mechanism of indexing
- List structures
- Passing arguments to a function
- Dynamic allocation
- Often associated with NULL
- NULL is a value of pointer that points to nothing (nothing is an invalid address)
- Attention CAPITAL letters!!



2 Pointer Definition and Operations

Live pacing: about 8 min

Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

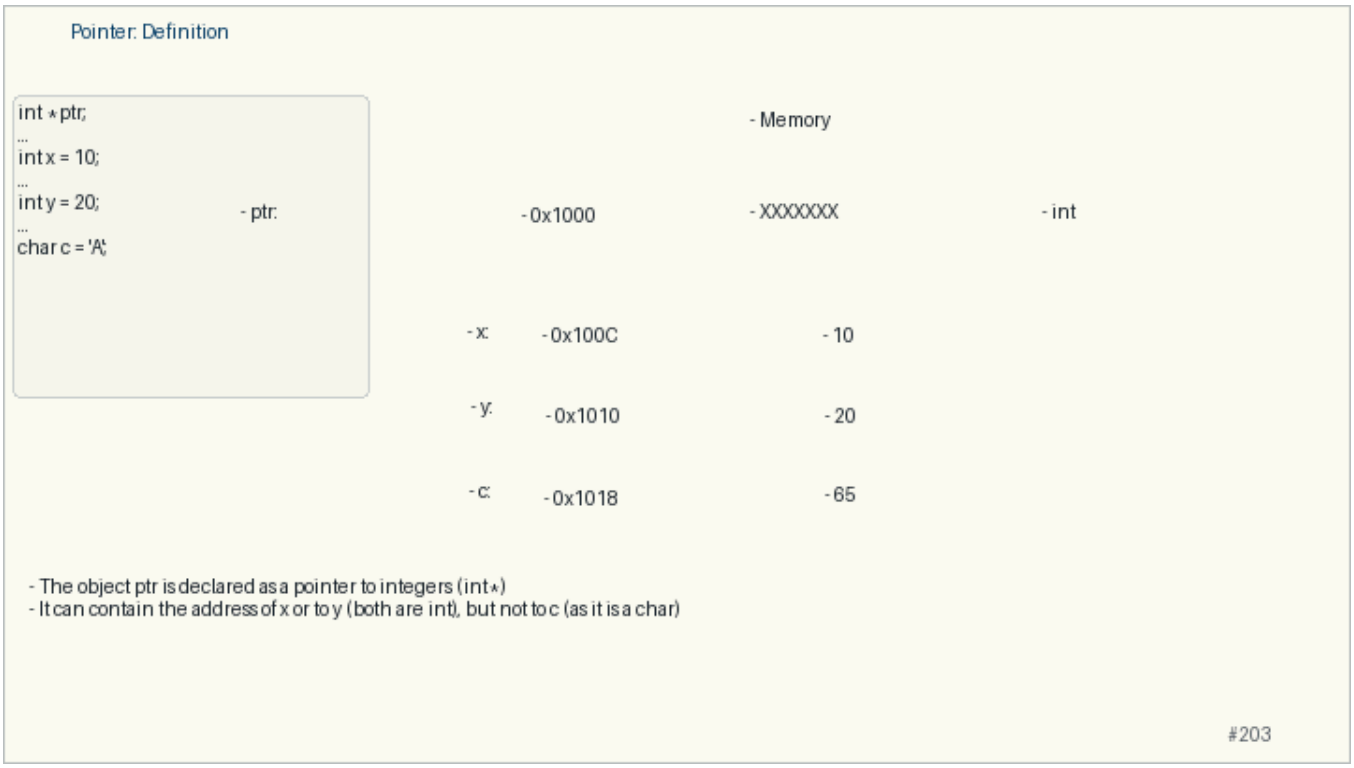
Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

2.1 Main Path

- Start from the question: What does a pointer value name?
- Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- Keep only this slide on the horizontal path during the live lecture.

2.2 Worked Example

- Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- Representative source slide: 203
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



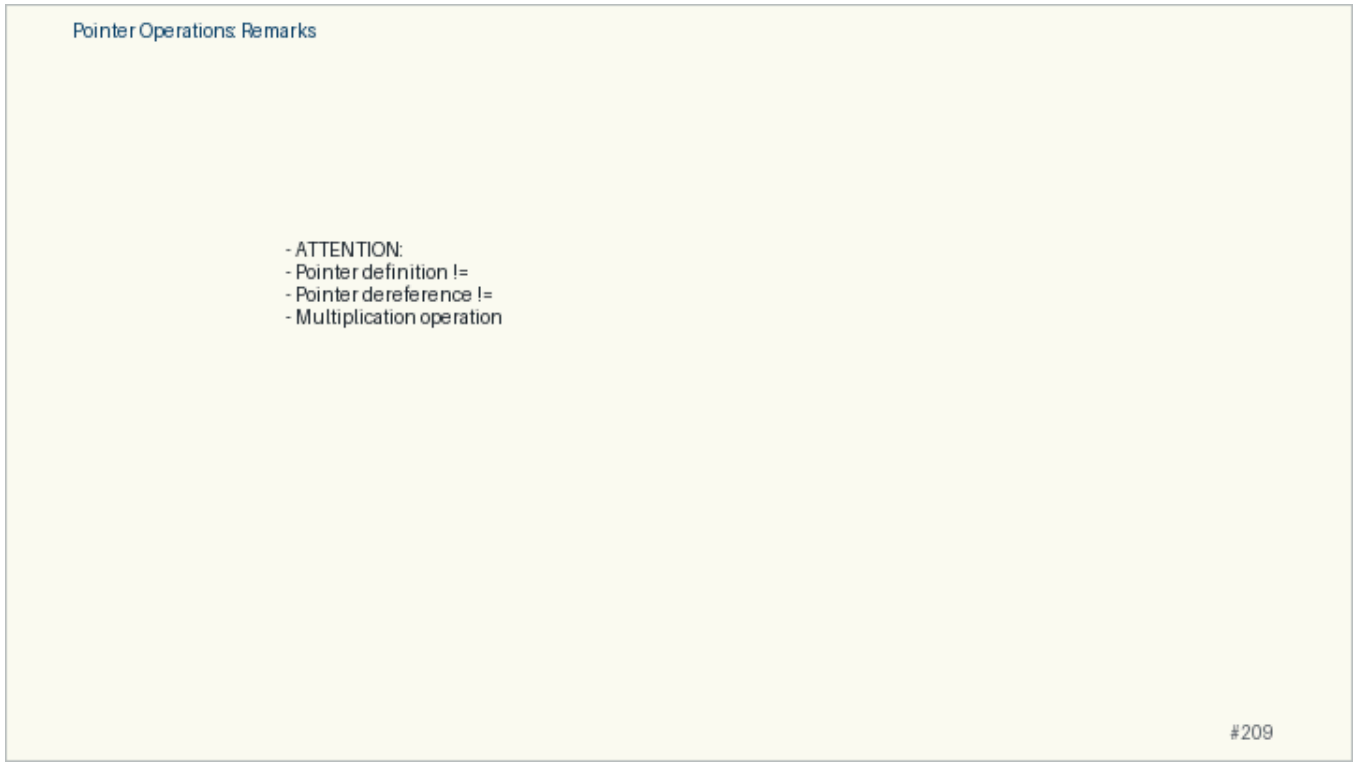
2.3 Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

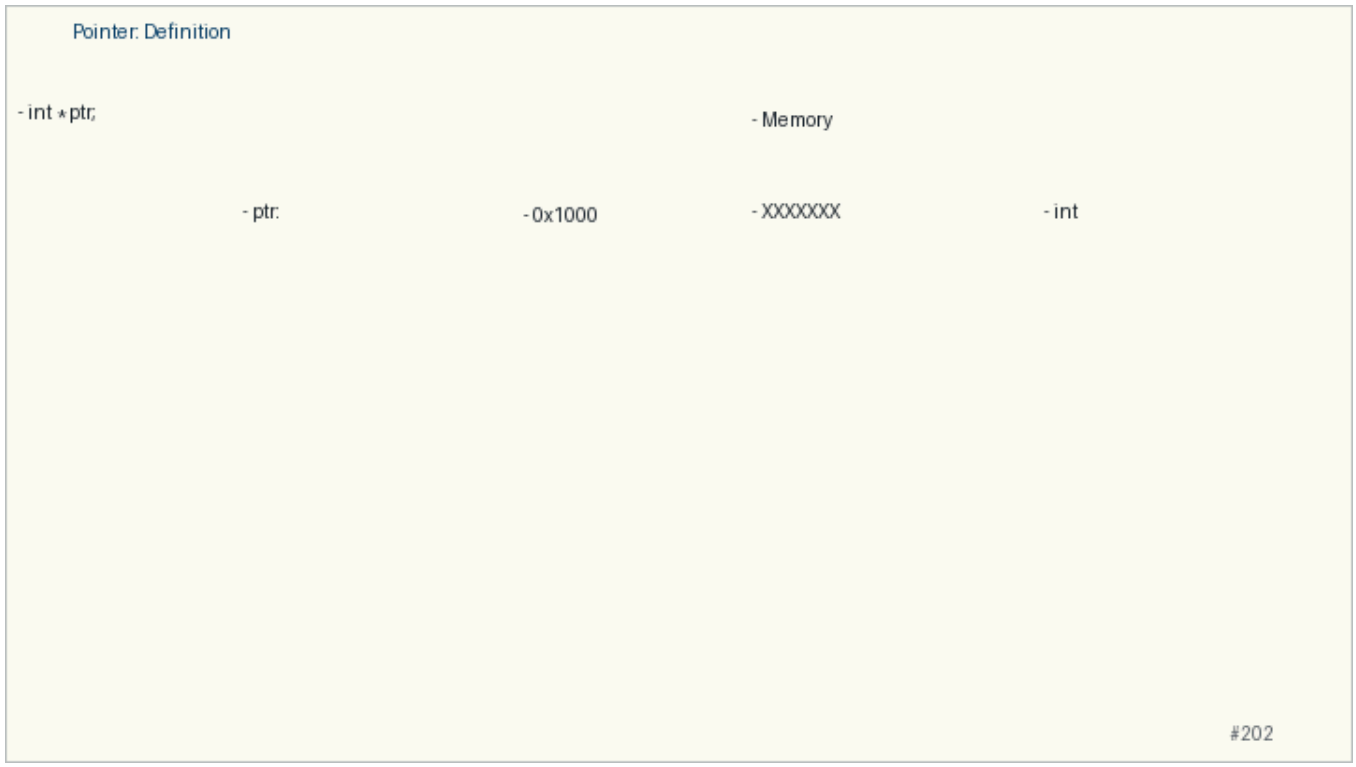
2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



2.5 Backup Index

Original slides 202-209



slide 202

Pointer: Definition

```
int * ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
```

- ptr:

- Memory

- ptr:	- 0x1000	- XXXXXXX	- int
- x:	- 0x100C	- 10	
- y:	- 0x1010	- 20	
- c:	- 0x1018	- 65	

- The object ptr is declared as a pointer to integers (int *)
- It can contain the address of x or to y (both are int), but not to c (as it is a char)

#203

slide 203

Pointer Operations: Address reception

```
int * ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = &x;
```

- ptr:

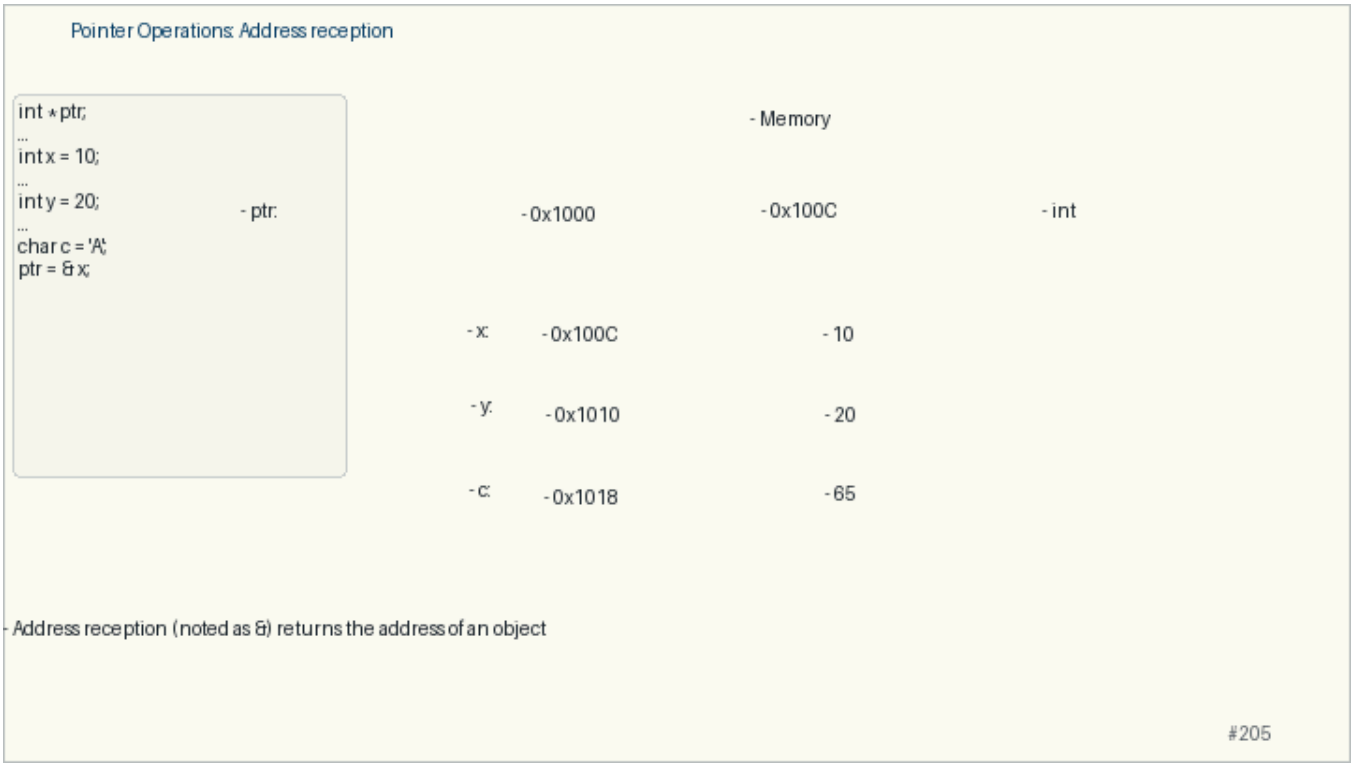
- Memory

- ptr:	- 0x1000	- XXXXXXX	- int
- x:	- 0x100C	- 10	
- y:	- 0x1010	- 20	
- c:	- 0x1018	- 65	

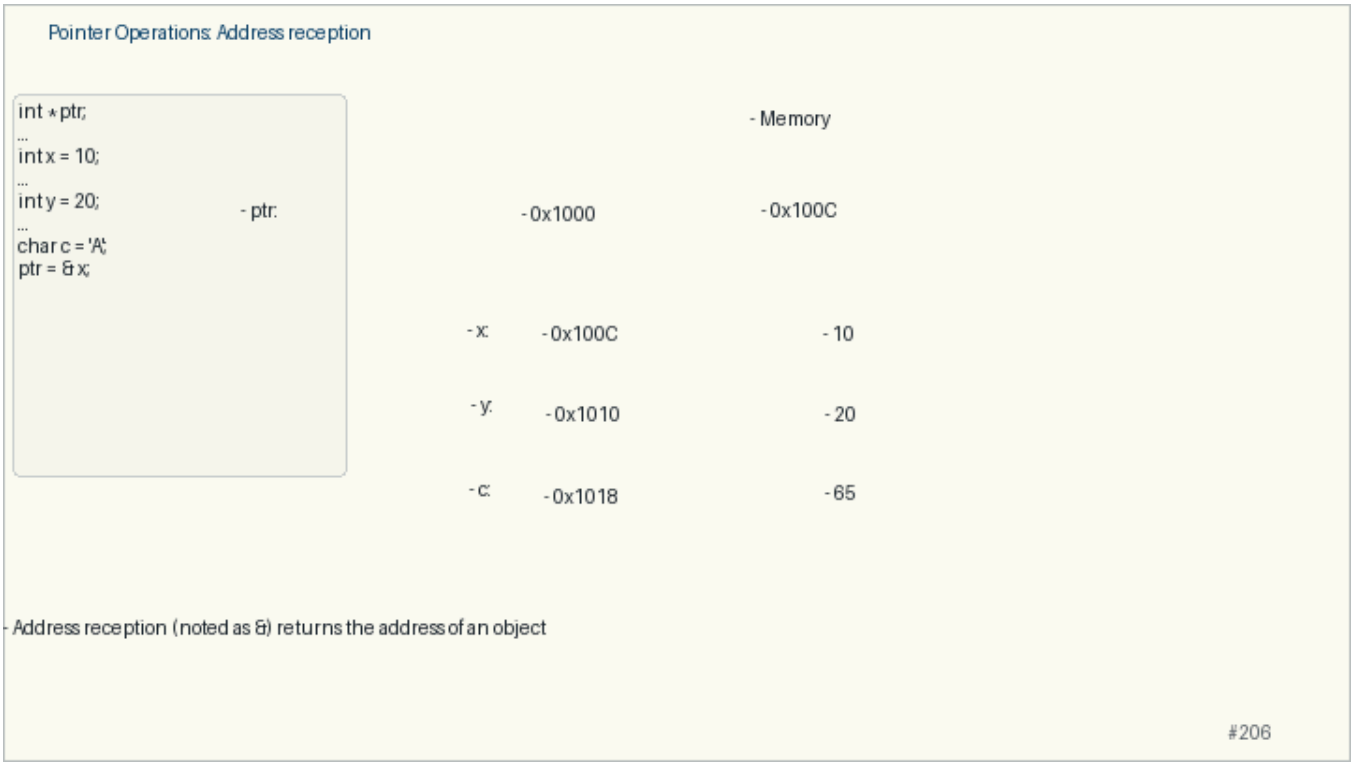
- Address reception (noted as &) returns the address of an object

#204

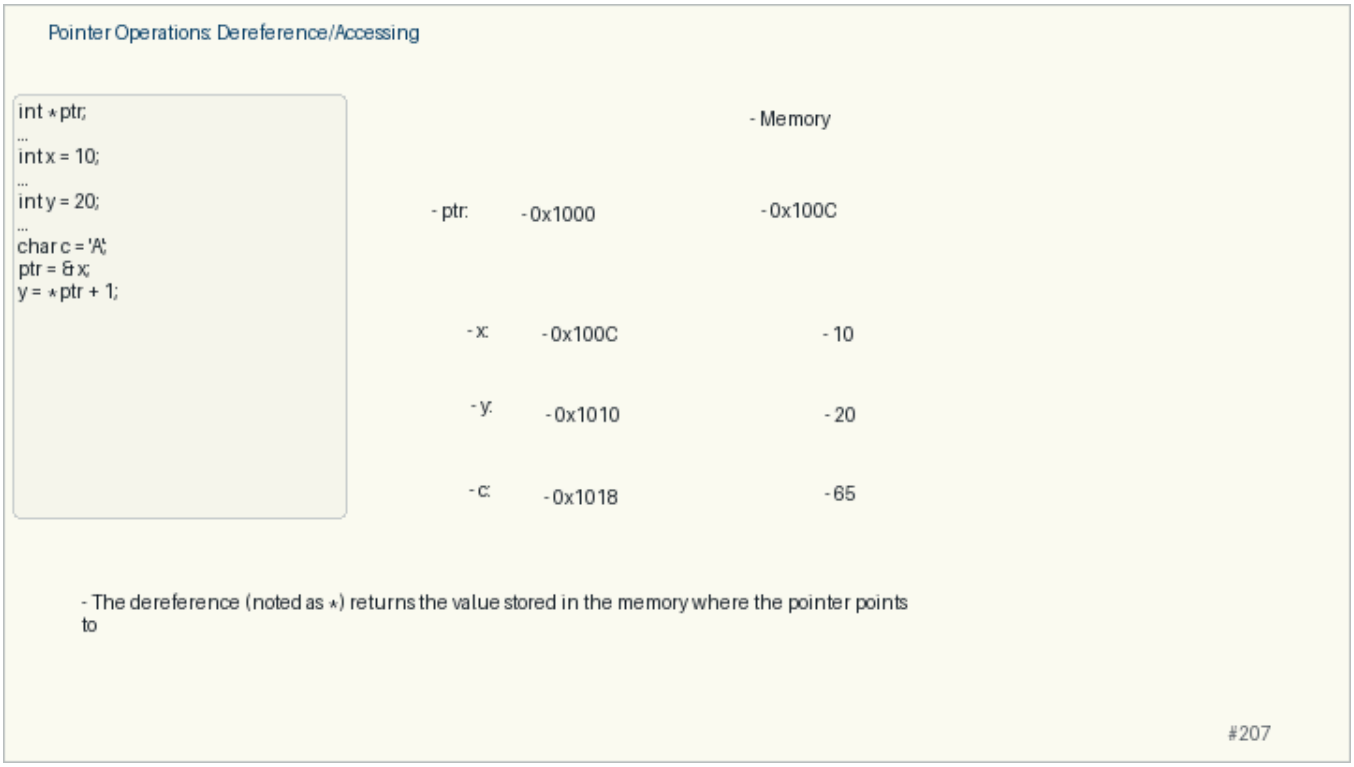
slide 204



slide 205



slide 206



slide 207

Anchors: Pointer: Definition; Pointer Operations: Address reception; Pointer Operations: Dereference/Accessing; Pointer Operations: Remarks

2.6 Backup: Pointer: Definition

Original slide 202

- int *ptr;
- Memory
- ptr:
- XXXXXXX
- int
- 0x1000

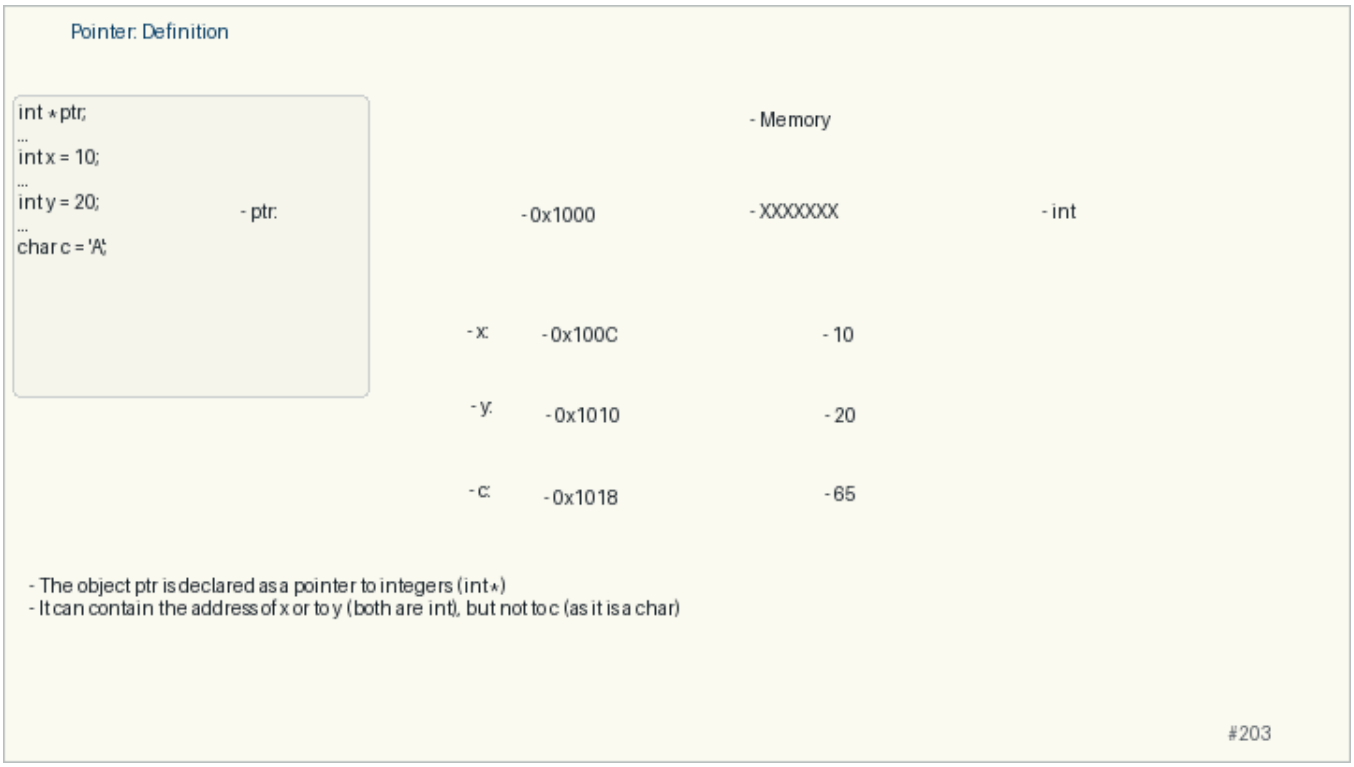
2.7 Backup: Pointer: Definition

Original slide 203

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
```

- Memory
- ptr:
- XXXXXXX
- int
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010
- 20
- c:
- 65
- 0x1018
- The object ptr is declared as a pointer to integers (int*)

- It can contain the address of x or to y (both are int), but not to c (as it is a char)



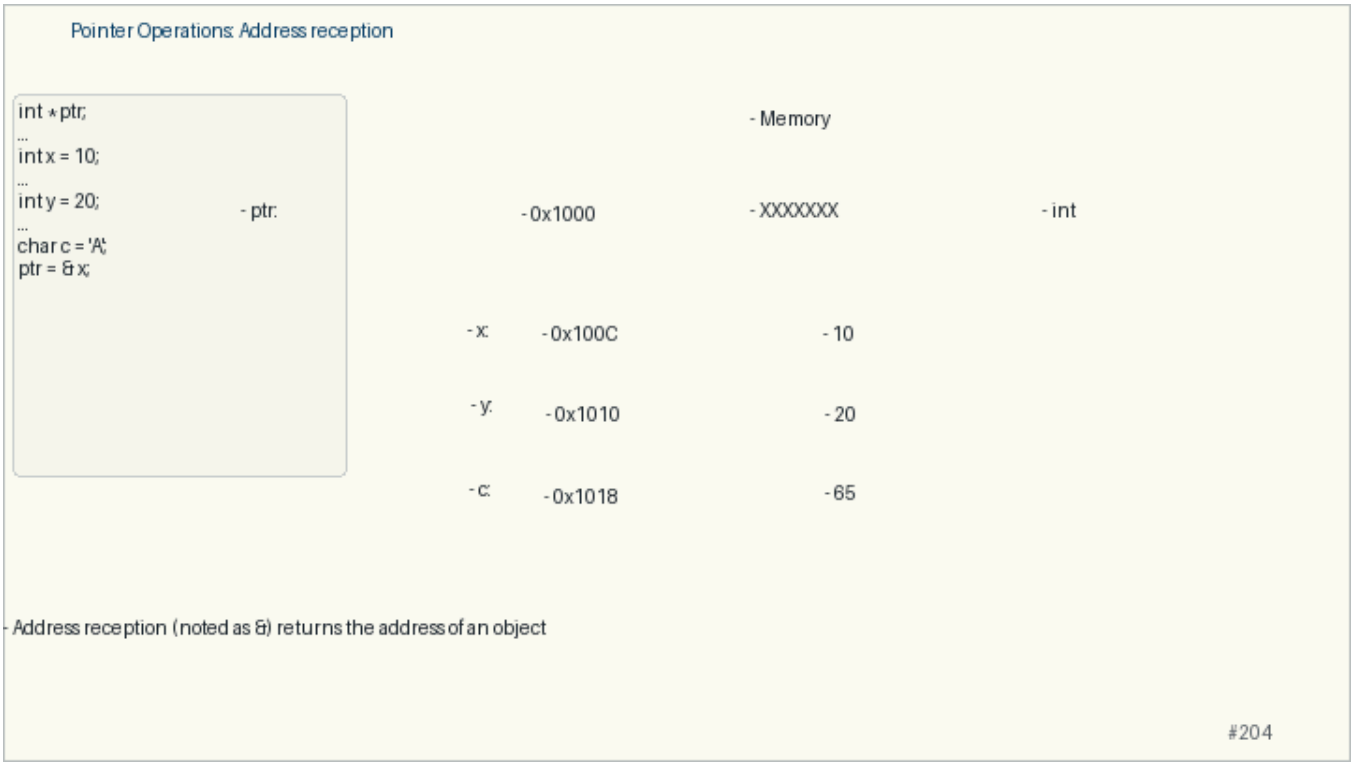
2.8 Backup: Pointer Operations: Address reception

Original slide 204

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = & x;
```

- Memory
- ptr:
- XXXXXXX
- int
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010
- 20
- c:
- 65

- 0x1018
- Address reception (noted as &) returns the address of an object



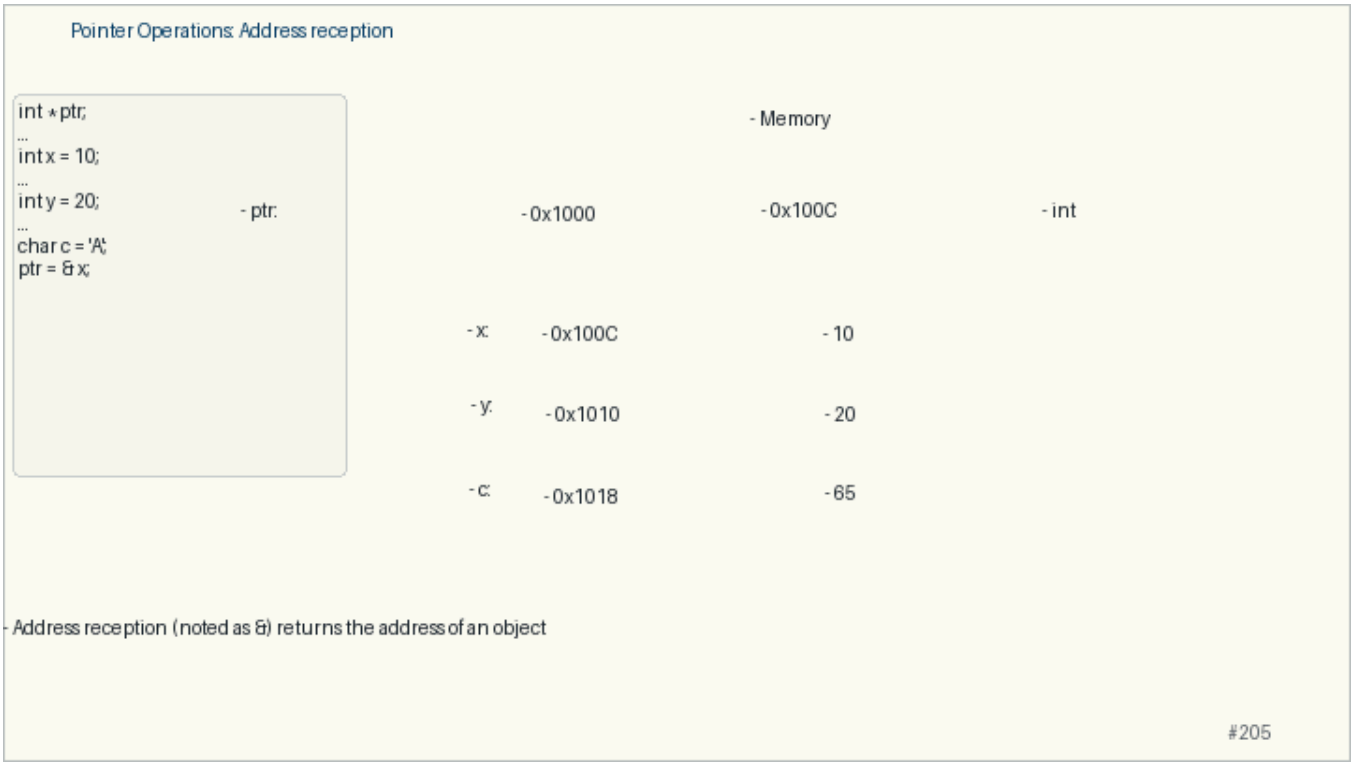
2.9 Backup: Pointer Operations: Address reception

Original slide 205

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = & x;
```

- Memory
- ptr:
- 0x100C
- int
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010
- 20
- c:

- 65
- 0x1018
- Address reception (noted as &) returns the address of an object



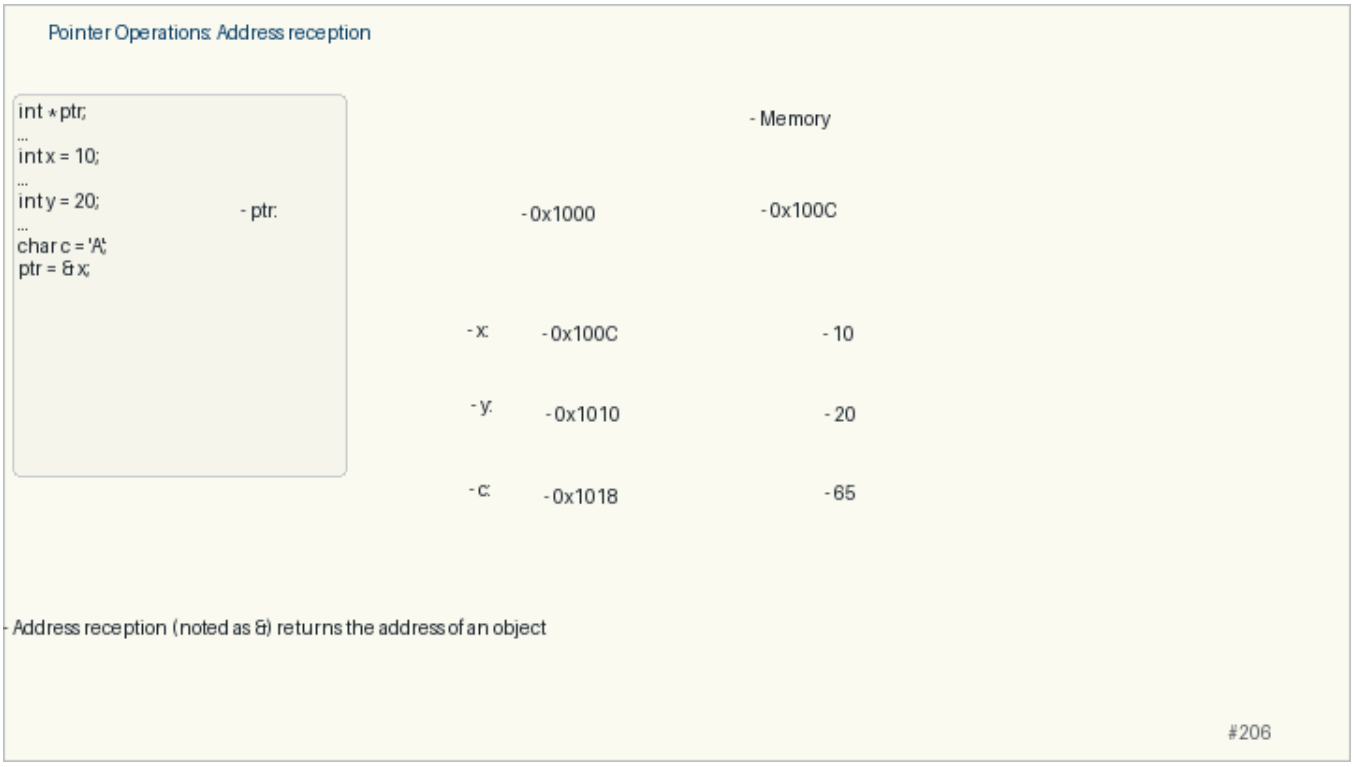
2.10 Backup: Pointer Operations: Address reception

Original slide 206

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = & x;
```

- Memory
- ptr:
- 0x100C
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010
- 20
- c:

- 65
- 0x1018
- Address reception (noted as &) returns the address of an object



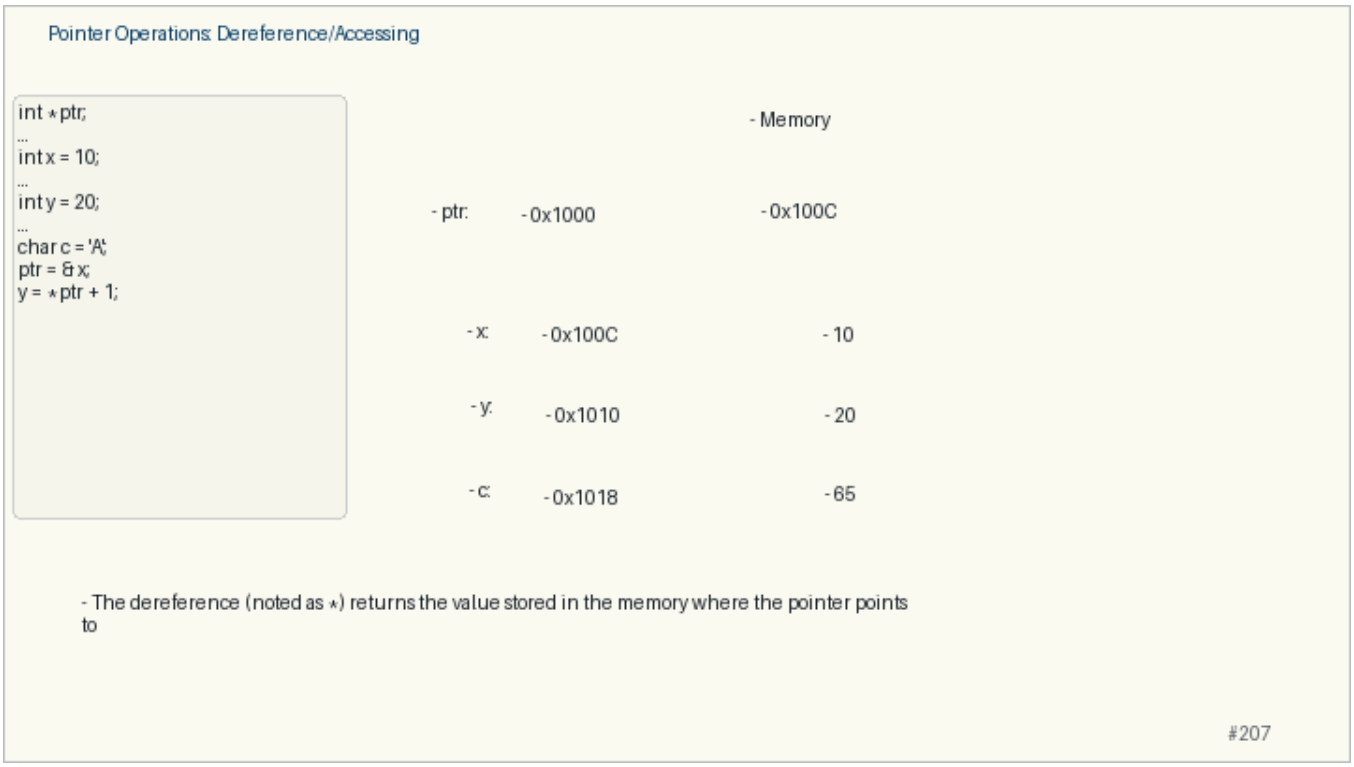
2.11 Backup: Pointer Operations: Dereference/Accessing

Original slide 207

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = & x;
y = *ptr + 1;
```

- Memory
- ptr:
- 0x100C
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010
- 20

- c:
- 65
- 0x1018
- The dereference (noted as *) returns the value stored in the memory where the pointer points to



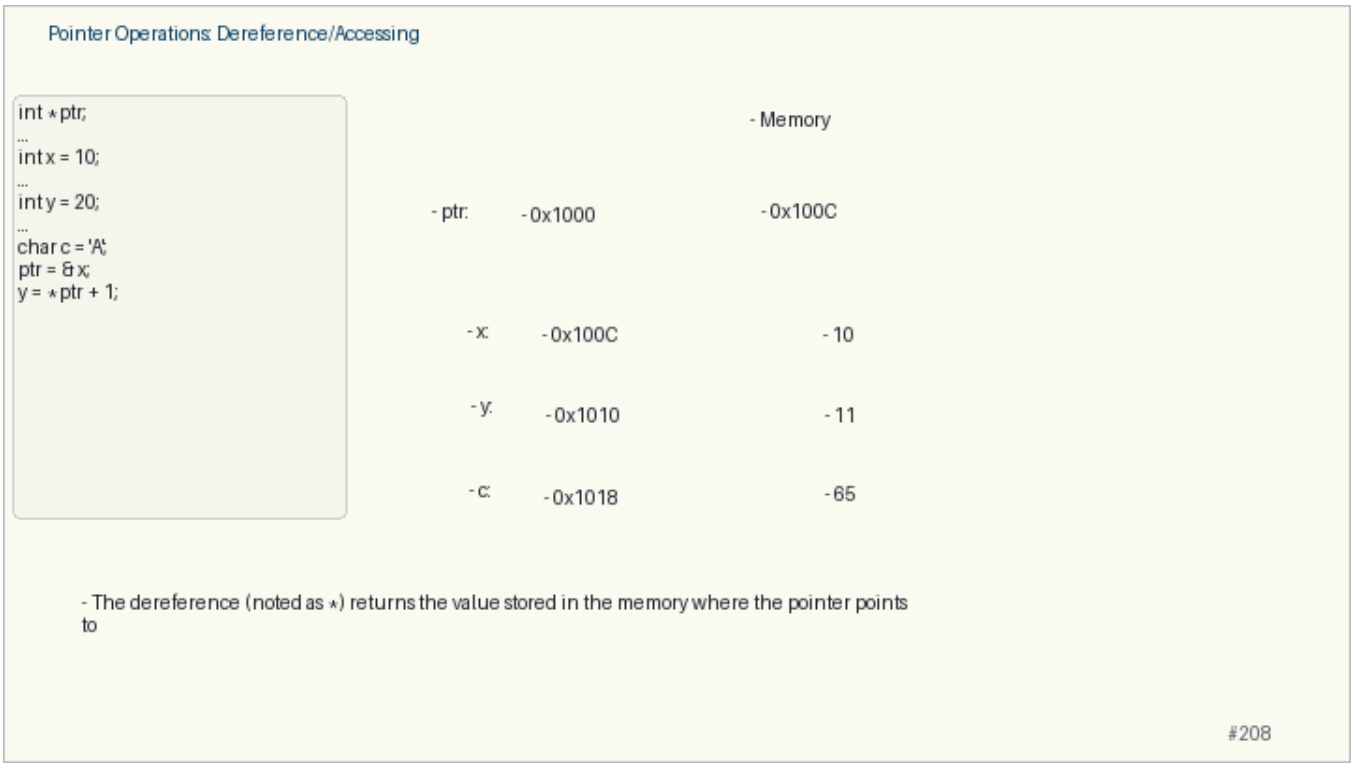
2.12 Backup: Pointer Operations: Dereference/Accessing

Original slide 208

```
int *ptr;
...
int x = 10;
...
int y = 20;
...
char c = 'A';
ptr = & x;
y = *ptr + 1;
```

- Memory
- ptr:
- 0x100C
- 0x1000
- x:
- 0x100C
- 10
- y:
- 0x1010

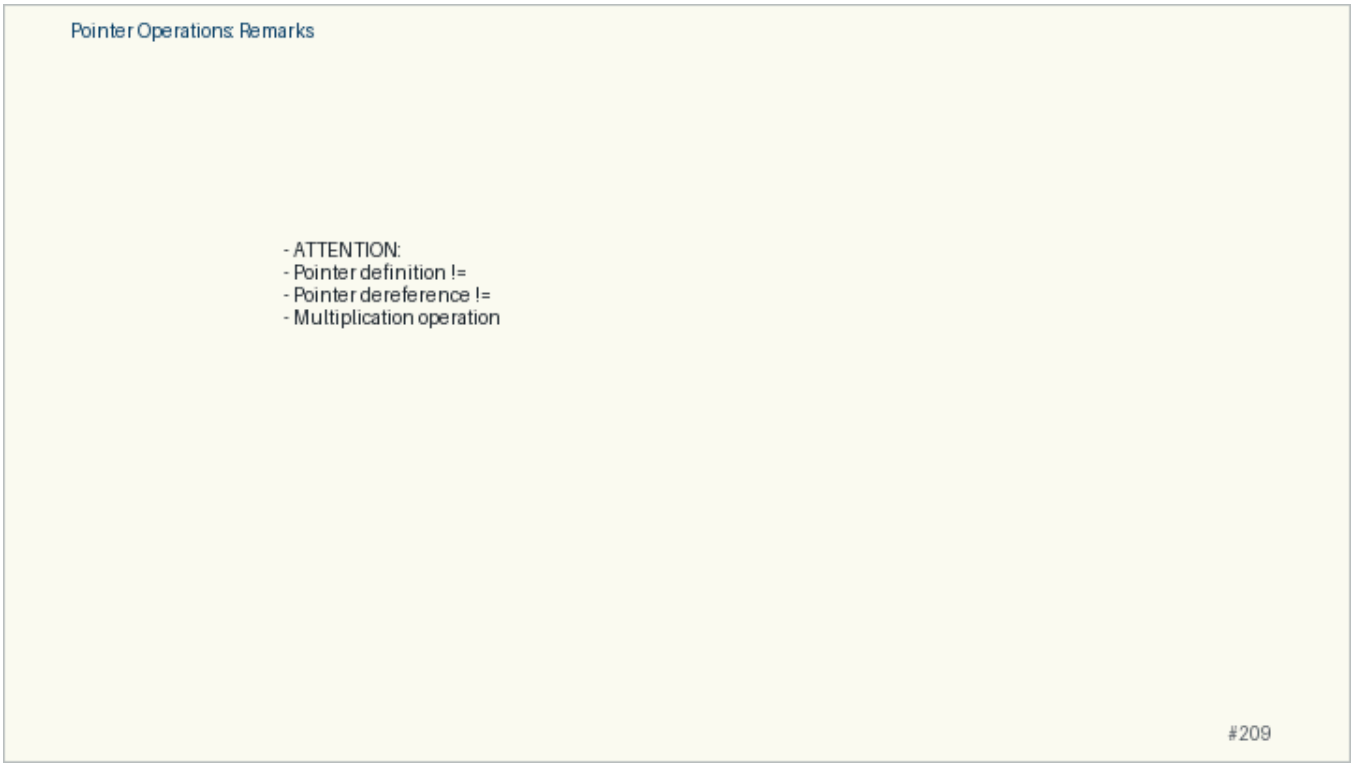
- 11
- c:
- 65
- 0x1018
- The dereference (noted as *) returns the value stored in the memory where the pointer points to



2.13 Backup: Pointer Operations: Remarks

Original slide 209

- ATTENTION:
- Pointer definition !=
- Pointer dereference !=
- Multiplication operation



3 Pointer to Pointer

Live pacing: about 8 min

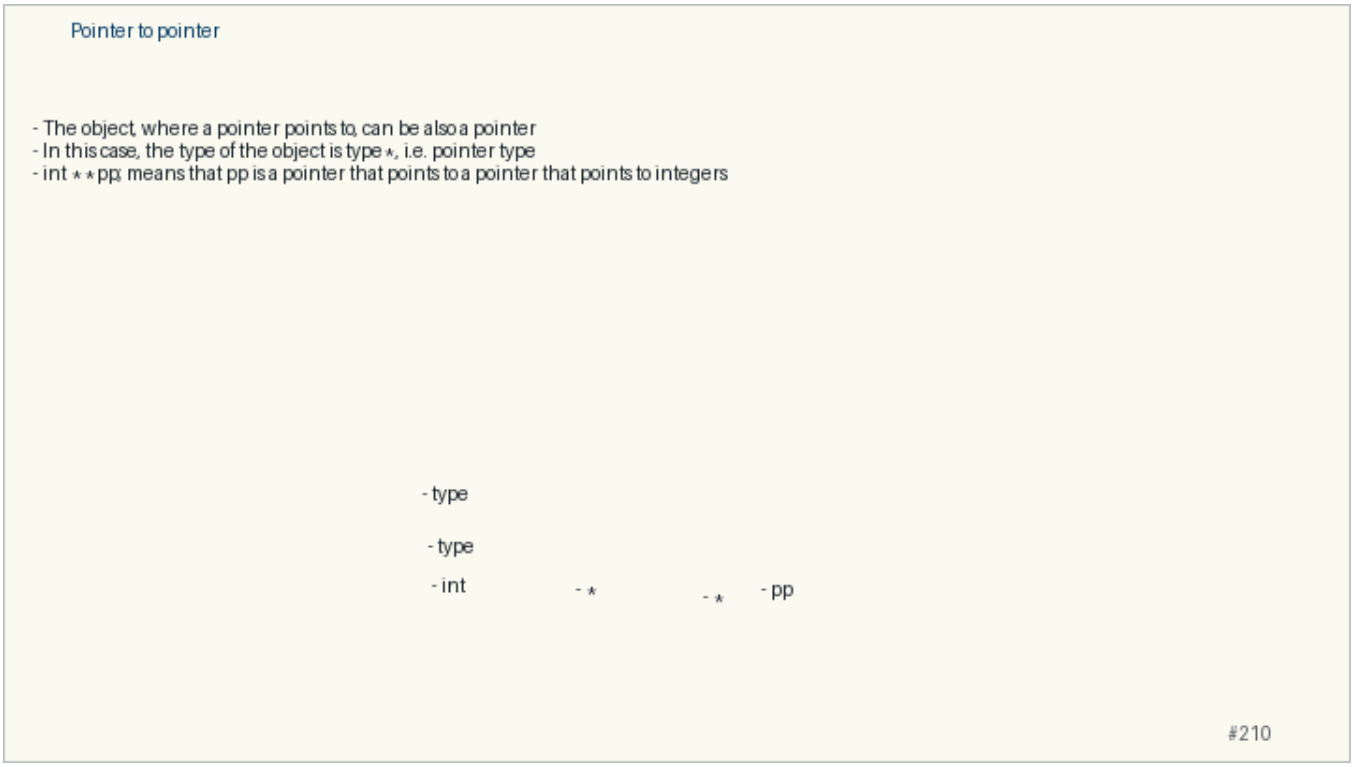
Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

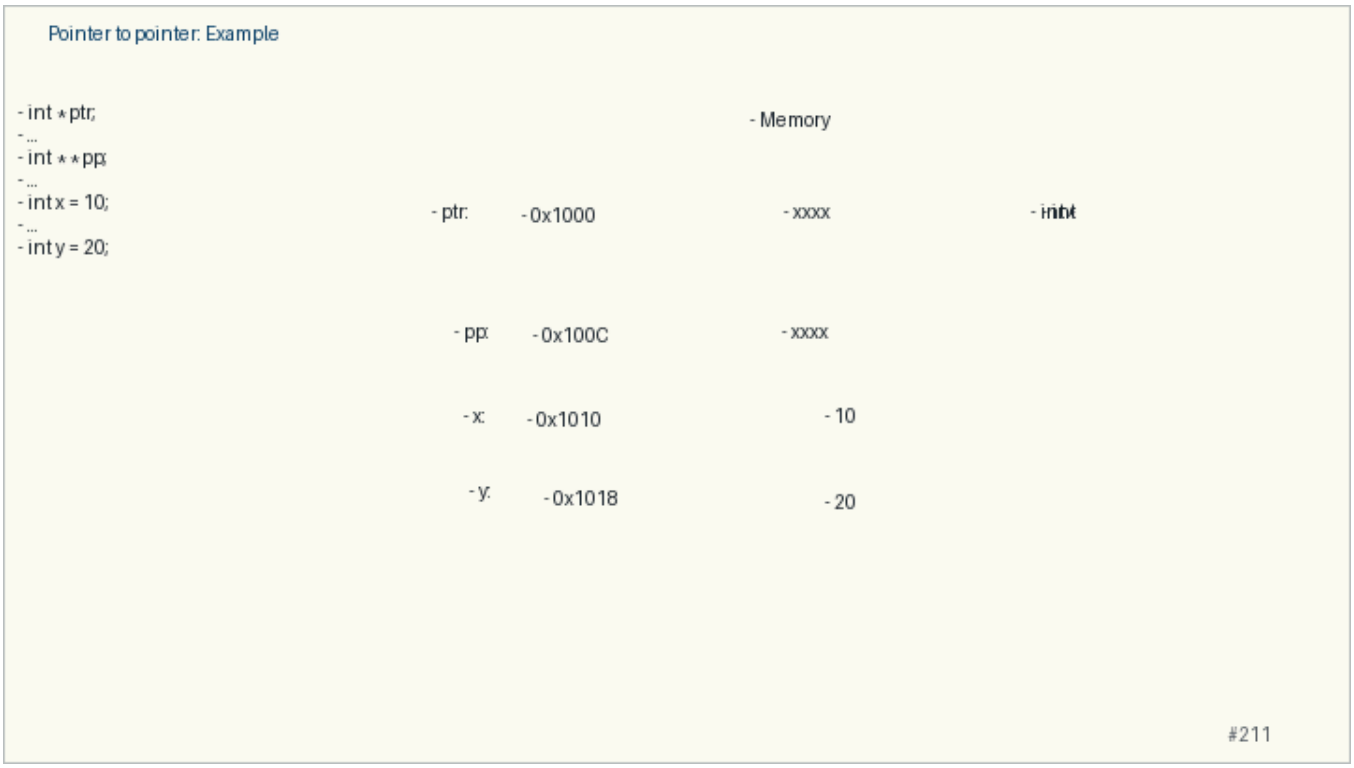
3.1 Main Path

- Start from the question: What does a pointer value name?
- Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- Keep only this slide on the horizontal path during the live lecture.



3.2 Worked Example

- Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- Representative source slide: 211
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



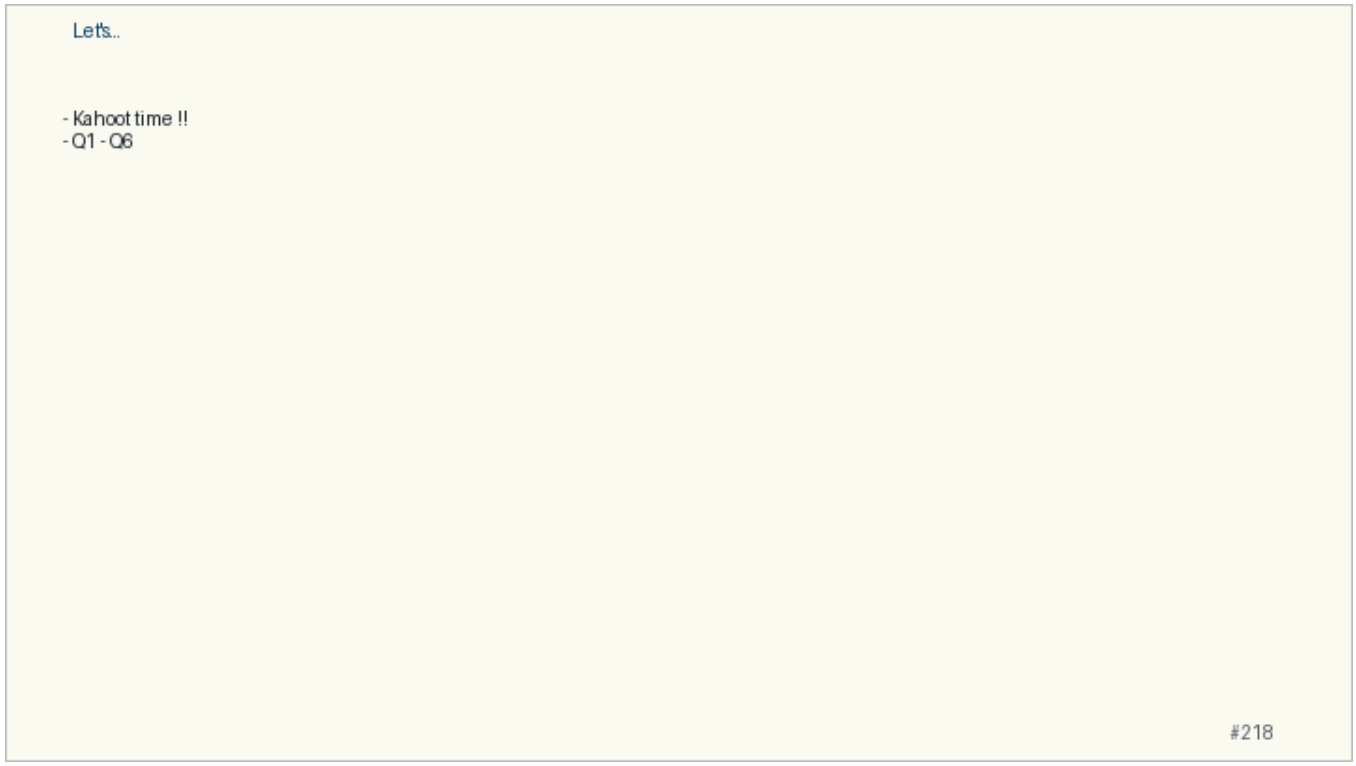
3.3 Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

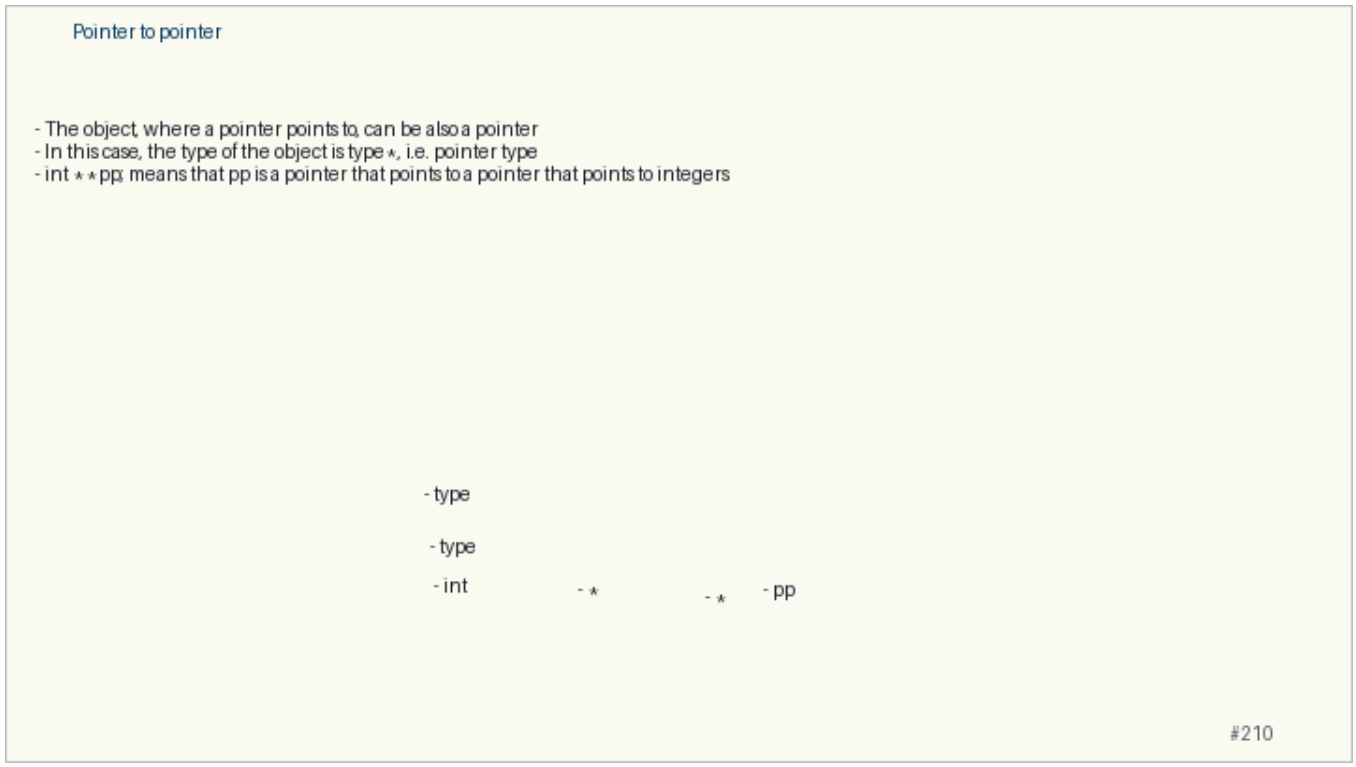
3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

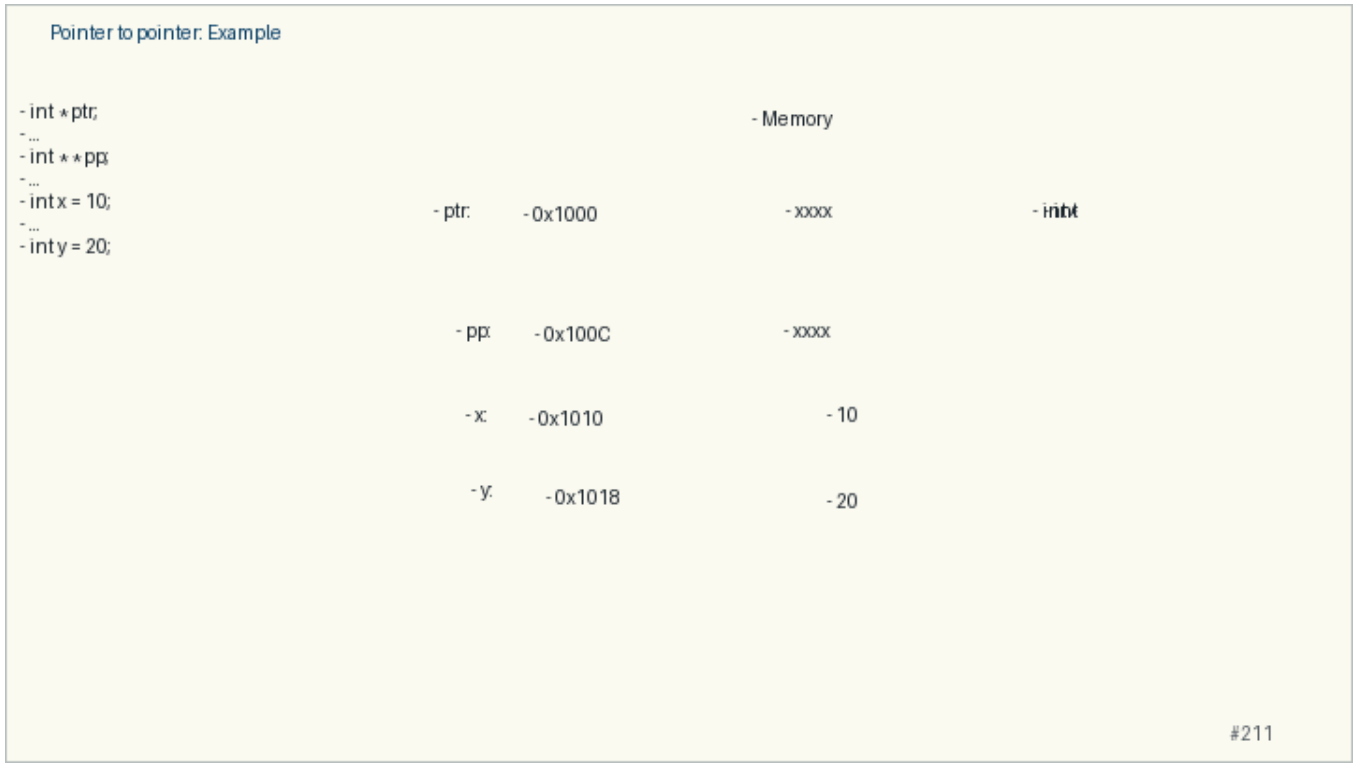


3.5 Backup Index

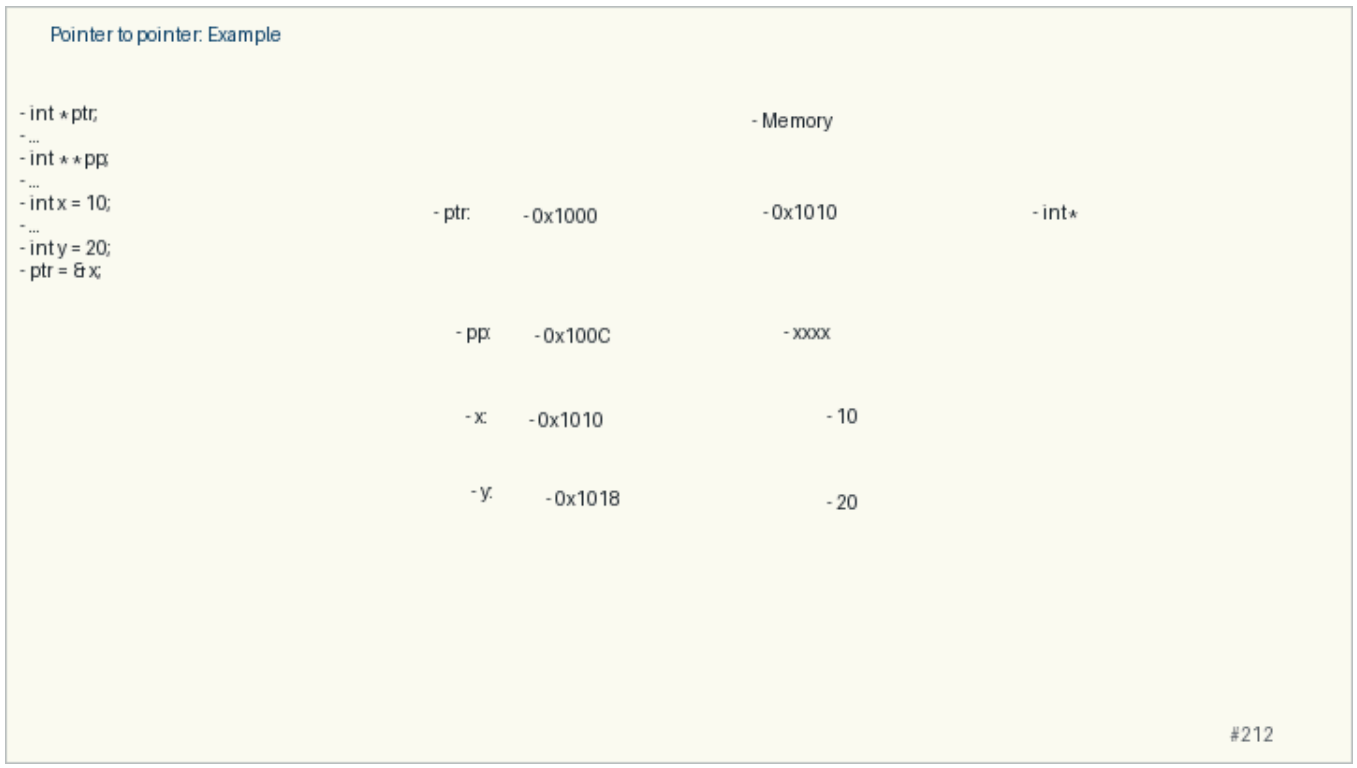
Original slides 210-218



slide 210



slide 211



slide 212

Pointer to pointer: Example

```
- int *ptr;
- ...
- int **pp;
- ...
- int x = 10;
- ...
- int y = 20;
- ptr = &x;
- pp = &ptr;
```

		- Memory
- ptr:	- 0x1000	- 0x1010
- pp:	- 0x100C	- 0x1000
- x:	- 0x1010	- 10
- y:	- 0x1018	- 20

- What is the result of *ptr?

#213

slide 213

Pointer to pointer: Example

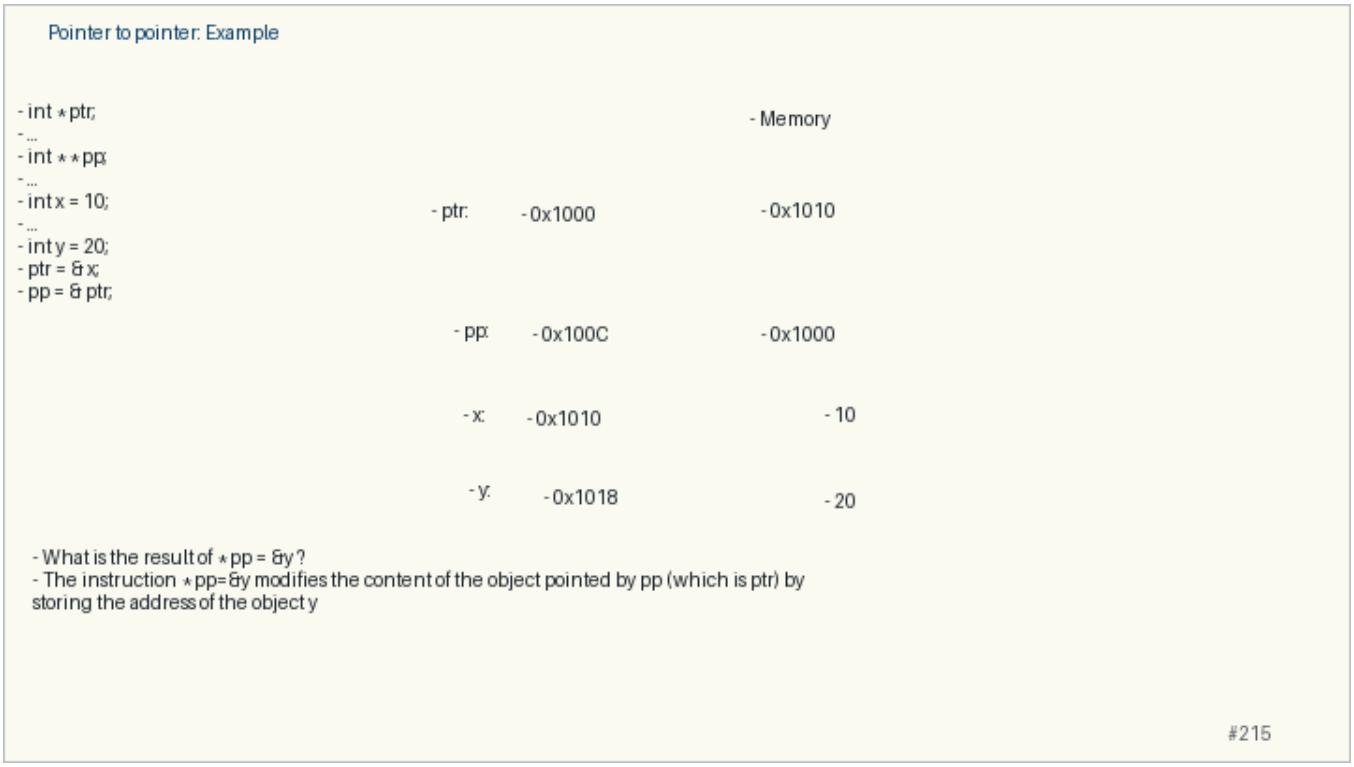
```
- int *ptr;
- ...
- int **pp;
- ...
- int x = 10;
- ...
- int y = 20;
- ptr = &x;
- pp = &ptr;
```

		- Memory
- ptr:	- 0x1000	- 0x1010
- pp:	- 0x100C	- 0x1000
- x:	- 0x1010	- 10
- y:	- 0x1018	- 20

- What is the result of *pp?

#214

slide 214



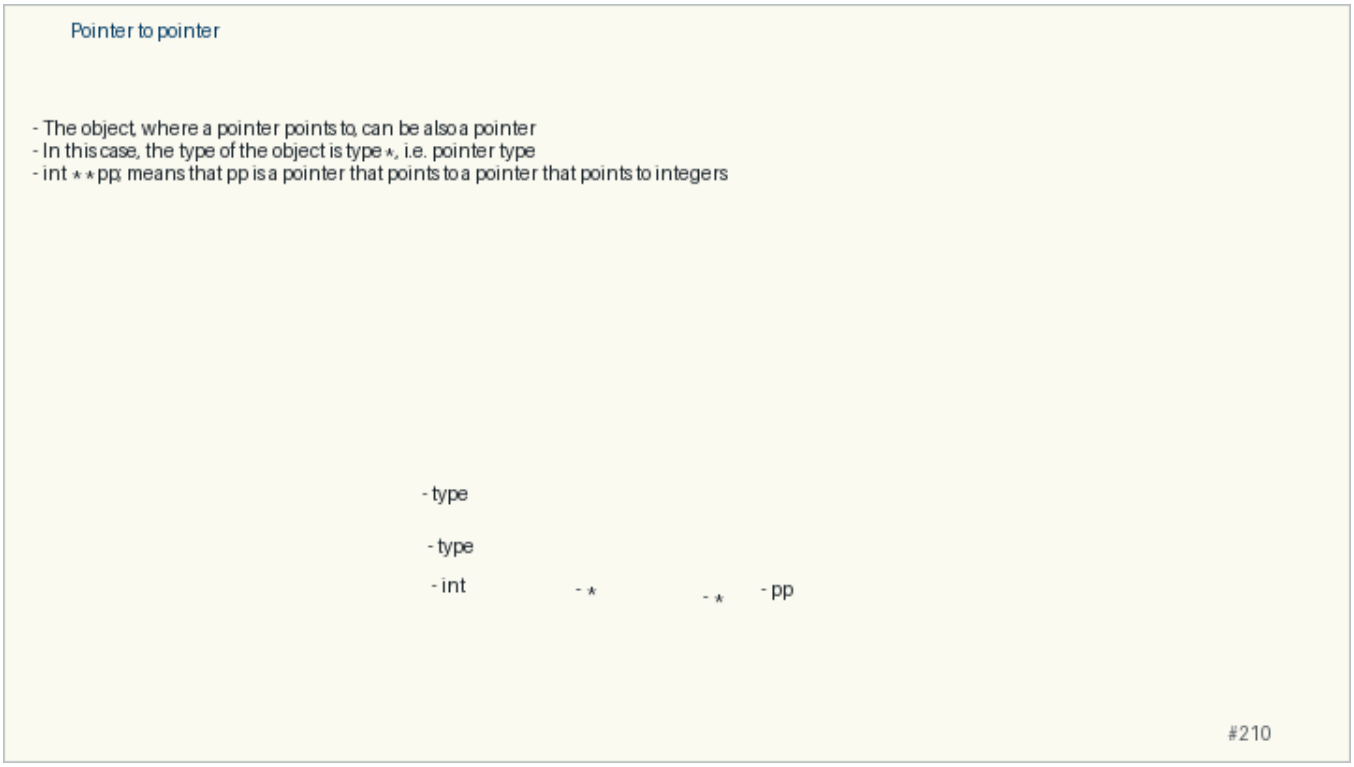
slide 215

Anchors: Pointer to pointer; Pointer to pointer: Example; Let's…

3.6 Backup: Pointer to pointer

Original slide 210

- The object, where a pointer points to, can be also a pointer
- In this case, the type of the object is type*, i.e. pointer type
- int **pp; means that pp is a pointer that points to a pointer that points to integers
- type
- type
- int
- *
- pp
- *

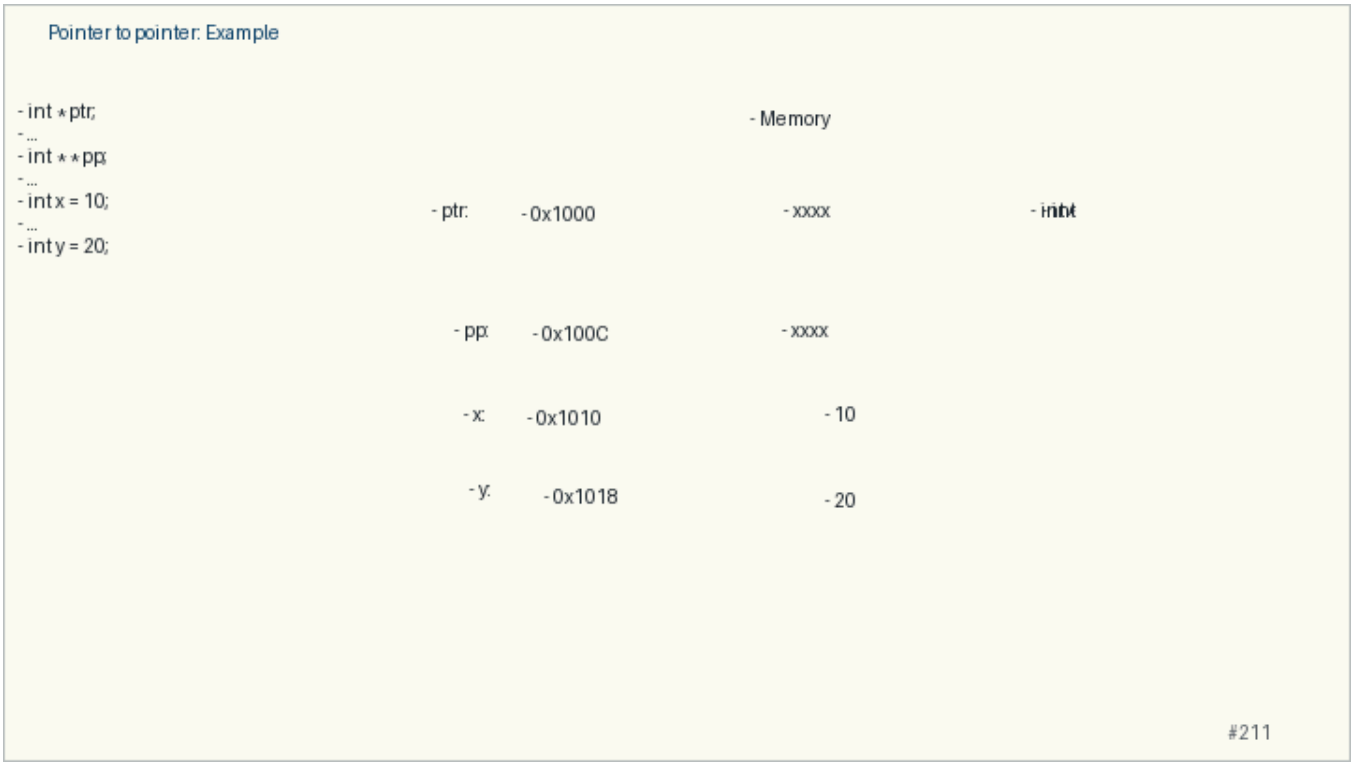


3.7 Backup: Pointer to pointer: Example

Original slide 211

- int *ptr;
- ...
- int **pp;
- ...
- int x = 10;
- ...
- int y = 20;
- Memory
- ptr:
- xxxx
- int*
- int
- 0x1000
- pp:
- xxxx
- 0x100C
- x:
- 10
- 0x1010
- y:

- 0x1018
- 20

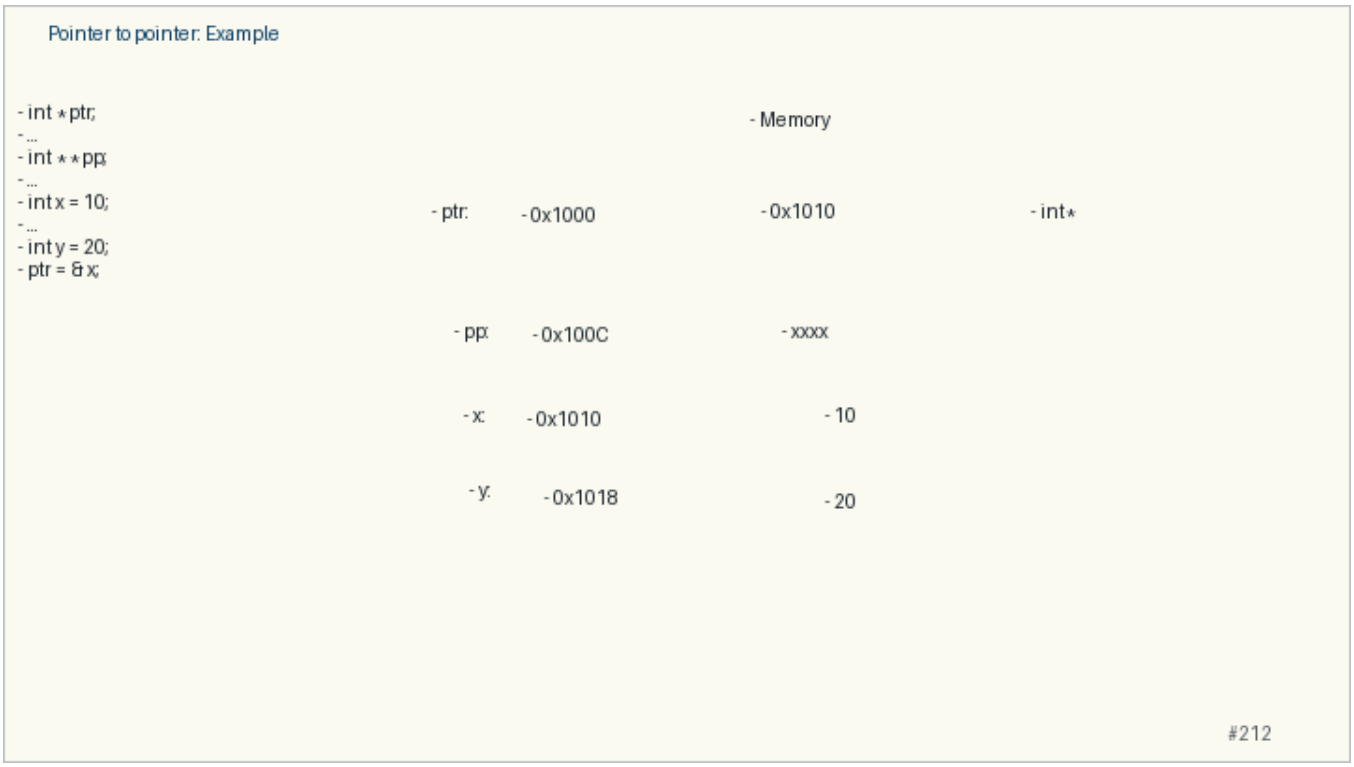


3.8 Backup: Pointer to pointer: Example

Original slide 212

- int *ptr;
- …
- int **pp;
- …
- int x = 10;
- …
- int y = 20;
- ptr = & x;
- Memory
- ptr:
- 0x1010
- int*
- 0x1000
- pp:
- xxxx
- 0x100C
- x:
- 10

- 0x1010
- y:
- 0x1018
- 20

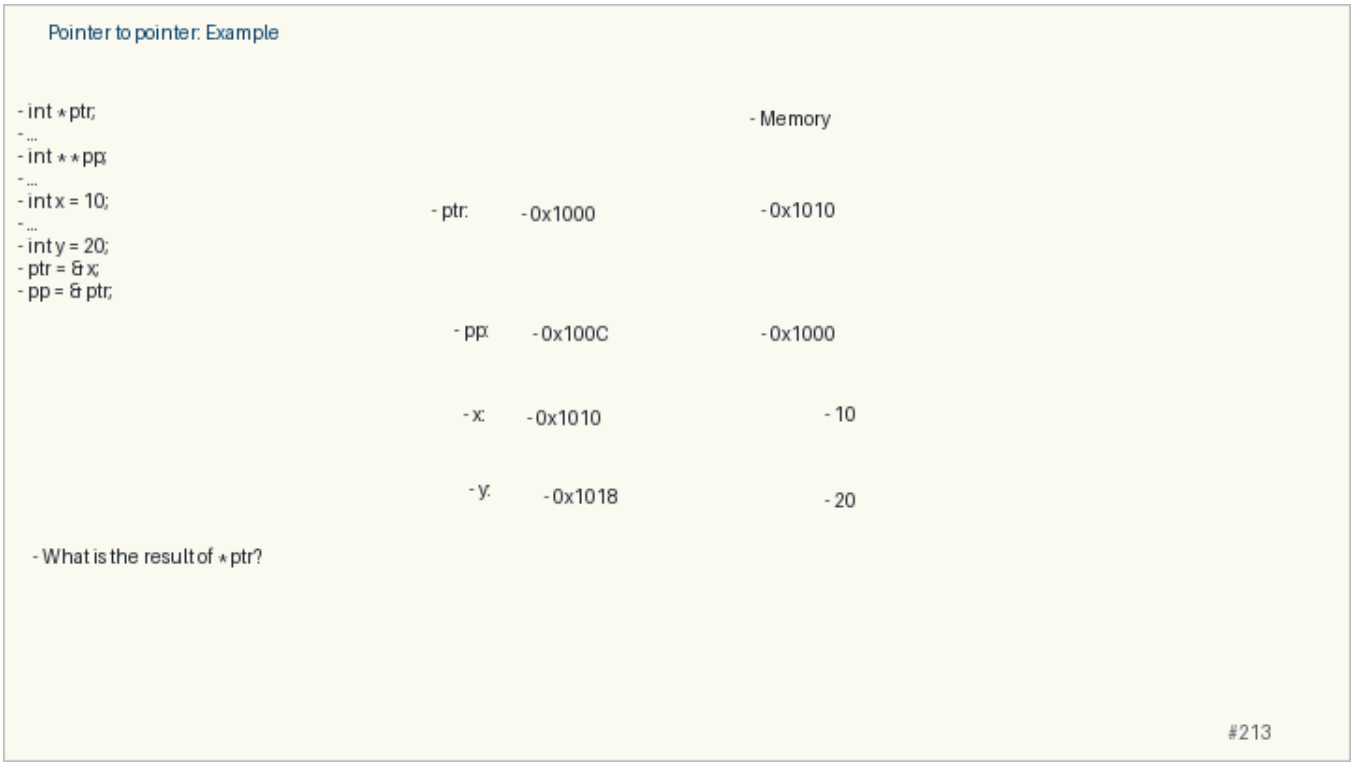


3.9 Backup: Pointer to pointer: Example

Original slide 213

- int *ptr;
- …
- int **pp;
- …
- int x = 10;
- …
- int y = 20;
- ptr = & x;
- pp = & ptr;
- Memory
- ptr:
- 0x1010
- 0x1000
- pp:
- 0x100C
- 0x1000

- x:
- 10
- 0x1010
- y:
- 0x1018
- 20
- What is the result of *ptr?

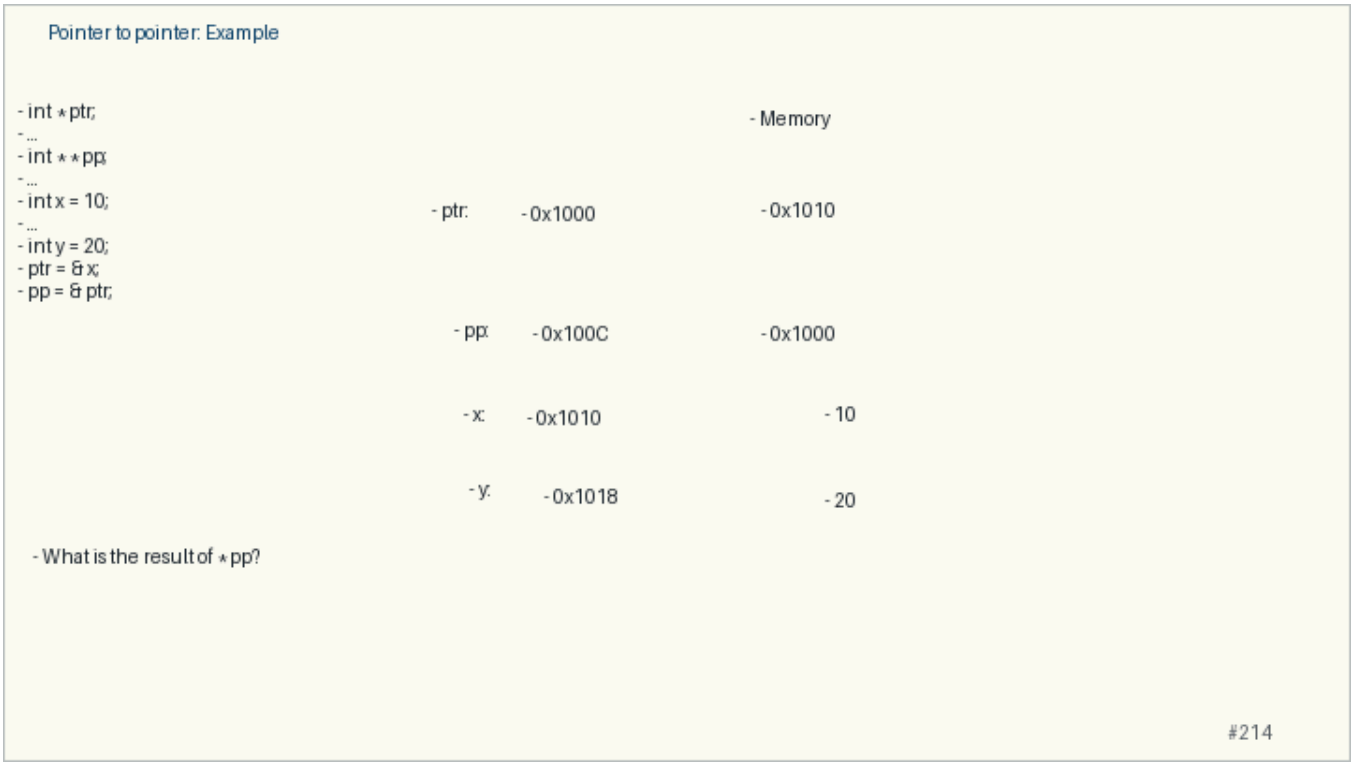


3.10 Backup: Pointer to pointer: Example

Original slide 214

- int *ptr;
- …
- int **pp;
- …
- int x = 10;
- …
- int y = 20;
- ptr = & x;
- pp = & ptr;
- Memory
- ptr:
- 0x1010
- 0x1000

- pp:
- 0x100C
- 0x1000
- x:
- 10
- 0x1010
- y:
- 0x1018
- 20
- What is the result of *pp?

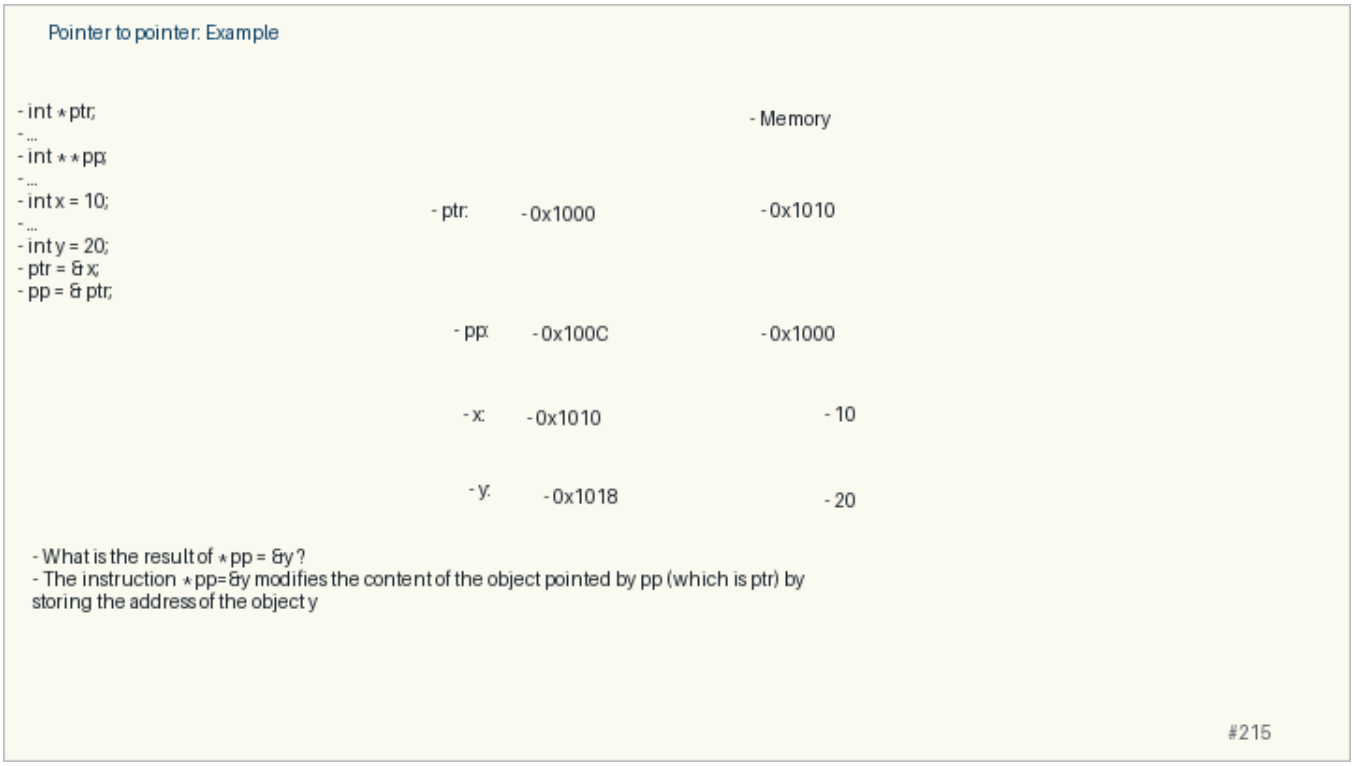


3.11 Backup: Pointer to pointer: Example

Original slide 215

- int *ptr;
- …
- int **pp;
- …
- int x = 10;
- …
- int y = 20;
- ptr = & x;
- pp = & ptr;
- Memory

- ptr:
- 0x1010
- 0x1000
- pp:
- 0x100C
- 0x1000
- x:
- 10
- 0x1010
- y:
- 0x1018
- 20
- What is the result of *pp = &y ?
- The instruction *pp=&y modifies the content of the object pointed by pp (which is ptr) by storing the address of the object y

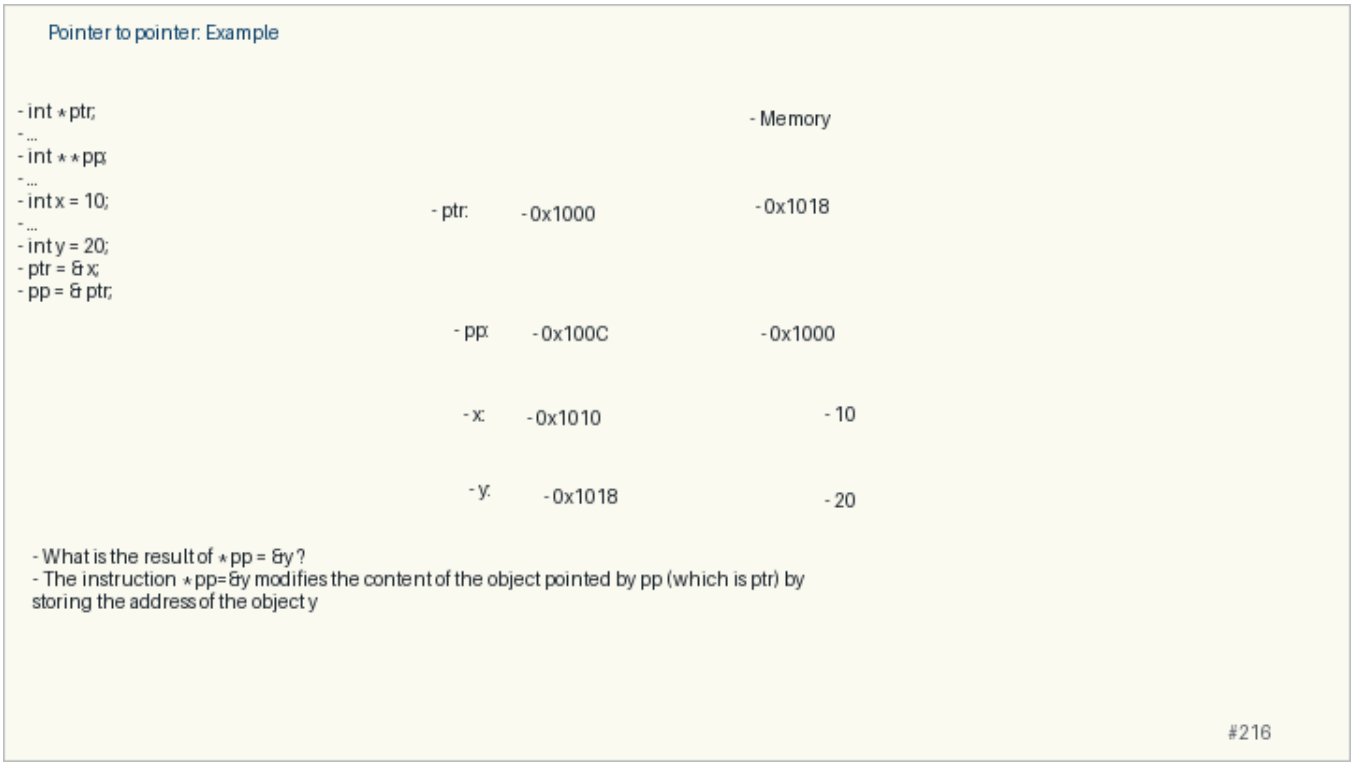


3.12 Backup: Pointer to pointer: Example

Original slide 216

- int *ptr;
- …
- int **pp;
- …
- int x = 10;
- …

- int y = 20;
- ptr = & x;
- pp = & ptr;
- Memory
- 0x1018
- ptr:
- 0x1000
- pp:
- 0x100C
- 0x1000
- x:
- 10
- 0x1010
- y:
- 0x1018
- 20
- What is the result of *pp = &y ?
- The instruction *pp=&y modifies the content of the object pointed by pp (which is ptr) by storing the address of the object y

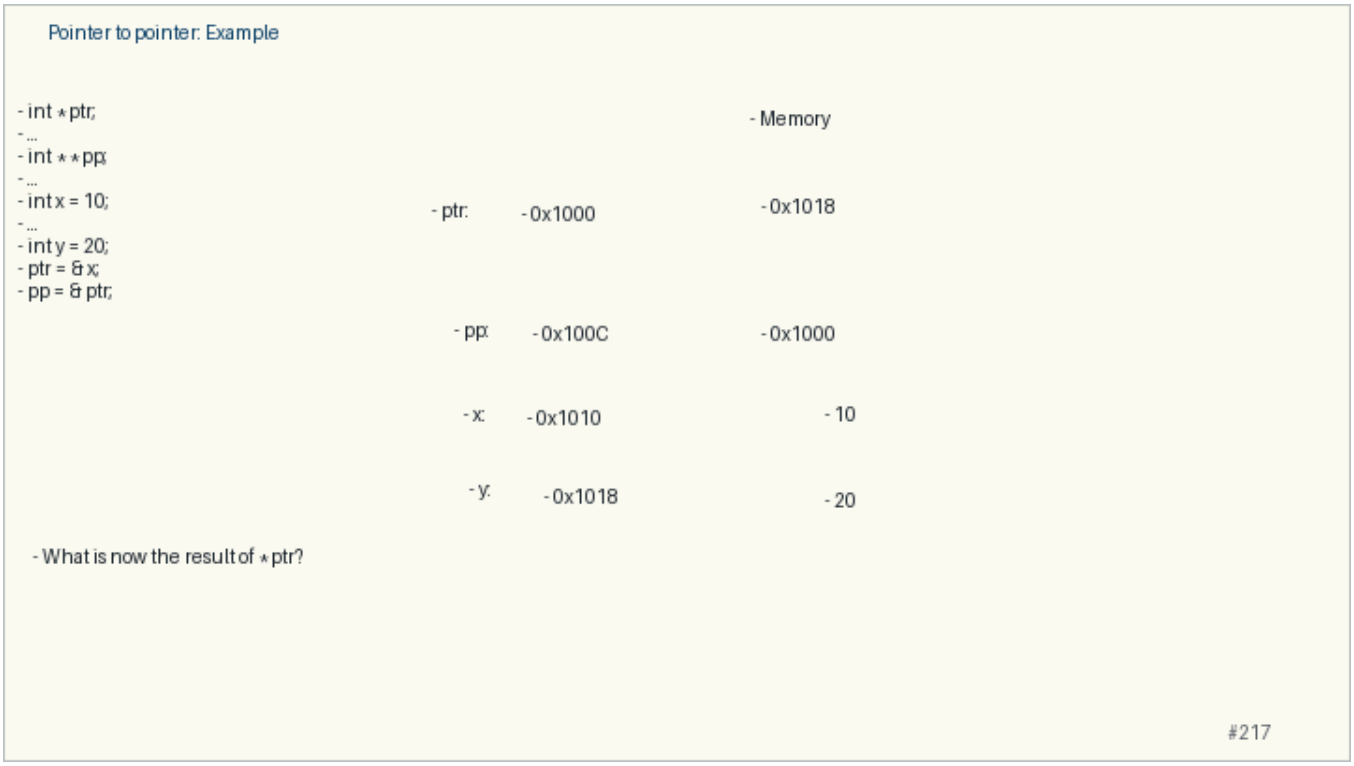


3.13 Backup: Pointer to pointer: Example

Original slide 217

- int *ptr;
- ...

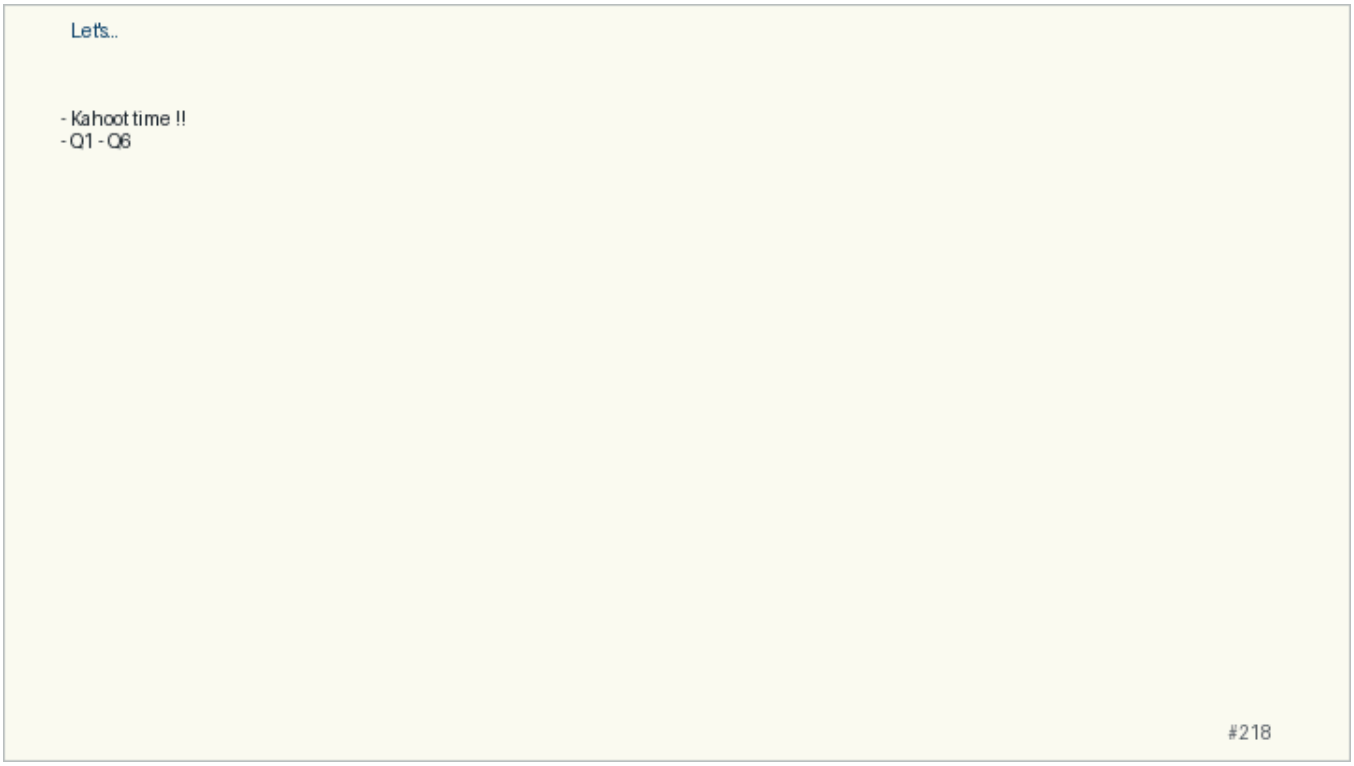
- int **pp;
- ...
- int x = 10;
- ...
- int y = 20;
- ptr = & x;
- pp = & ptr;
- Memory
- 0x1018
- ptr:
- 0x1000
- pp:
- 0x100C
- 0x1000
- x:
- 10
- 0x1010
- y:
- 0x1018
- 20
- What is now the result of *ptr?



3.14 Backup: Let's...

Original slide 218

- Kahoot time !!
- Q1 - Q6



4 Pointer Arithmetic

Live pacing: about 8 min

Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

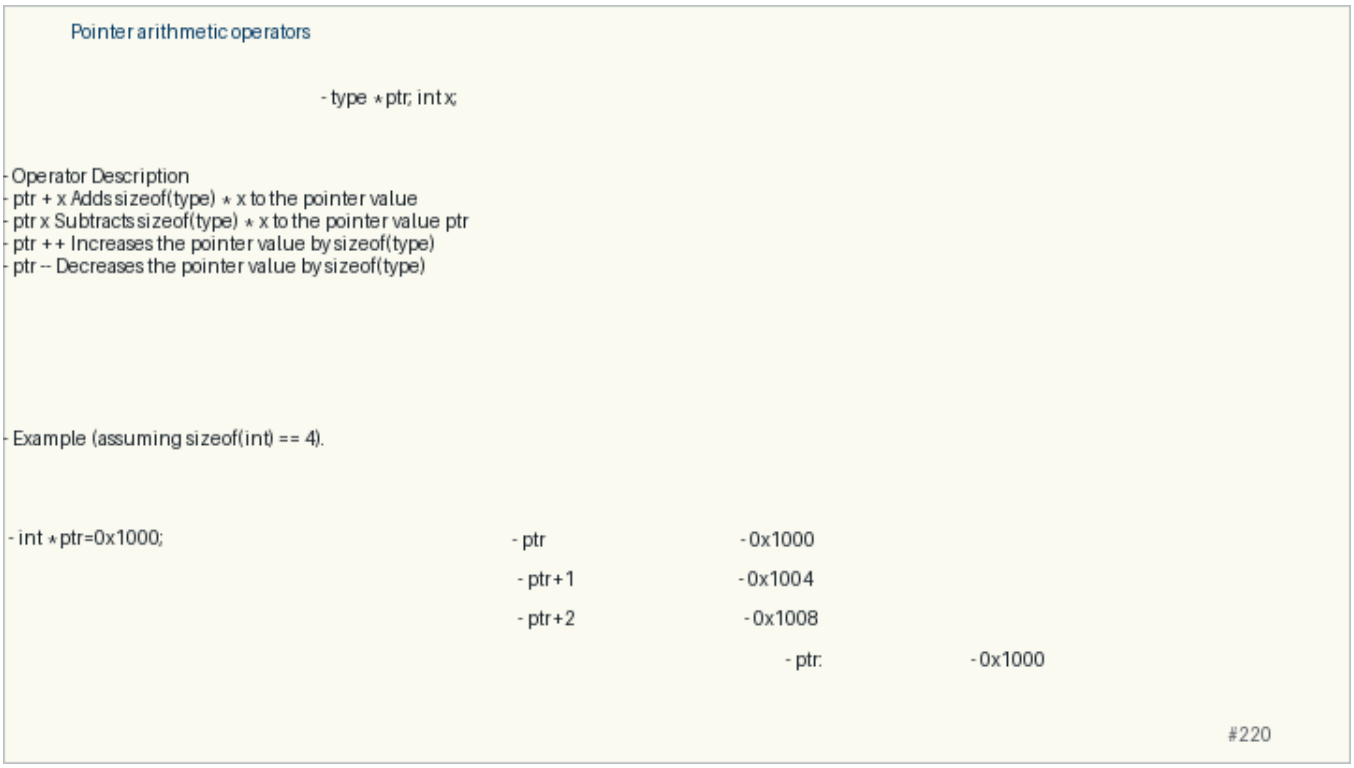
Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

4.1 Main Path

- Start from the question: What does a pointer value name?
- Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- Keep only this slide on the horizontal path during the live lecture.

4.2 Worked Example

- Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- Representative source slide: 220
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



4.3 Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

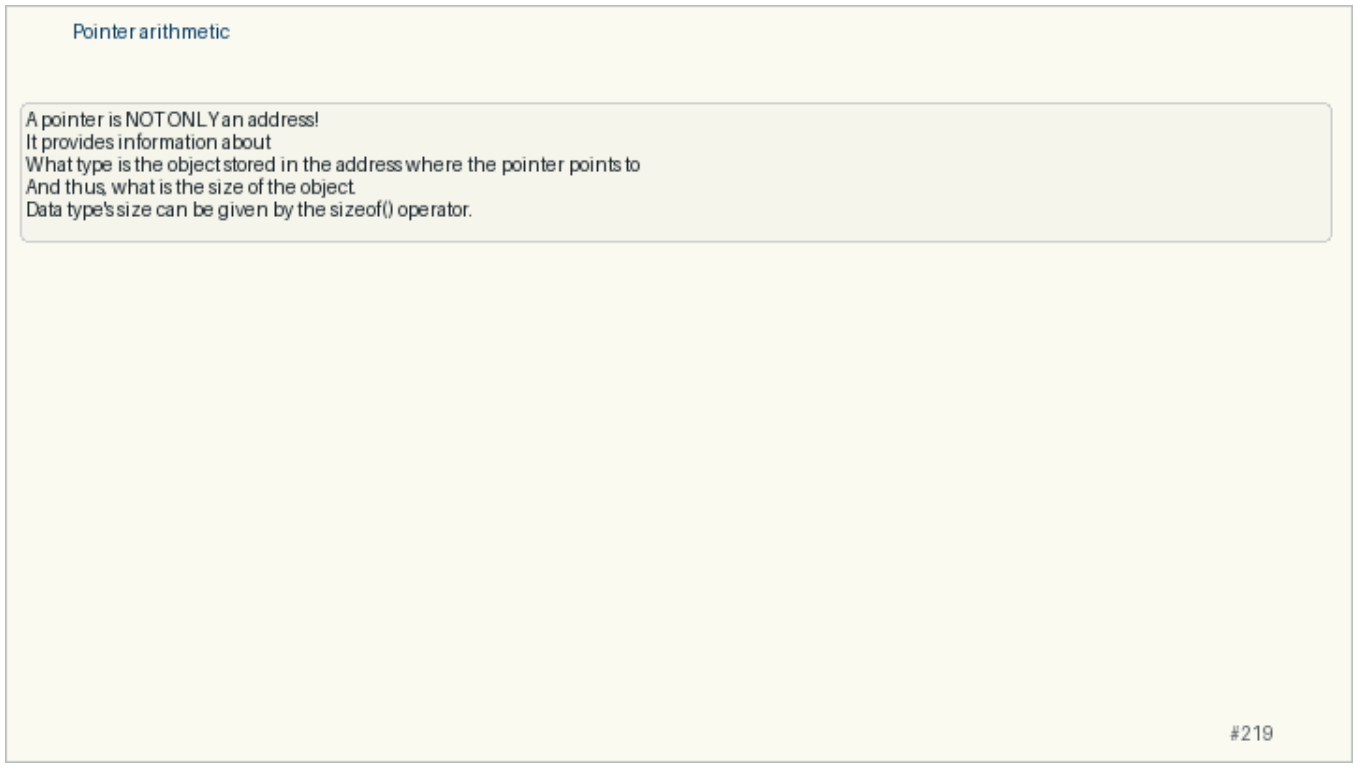
- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

4.4 Check Question

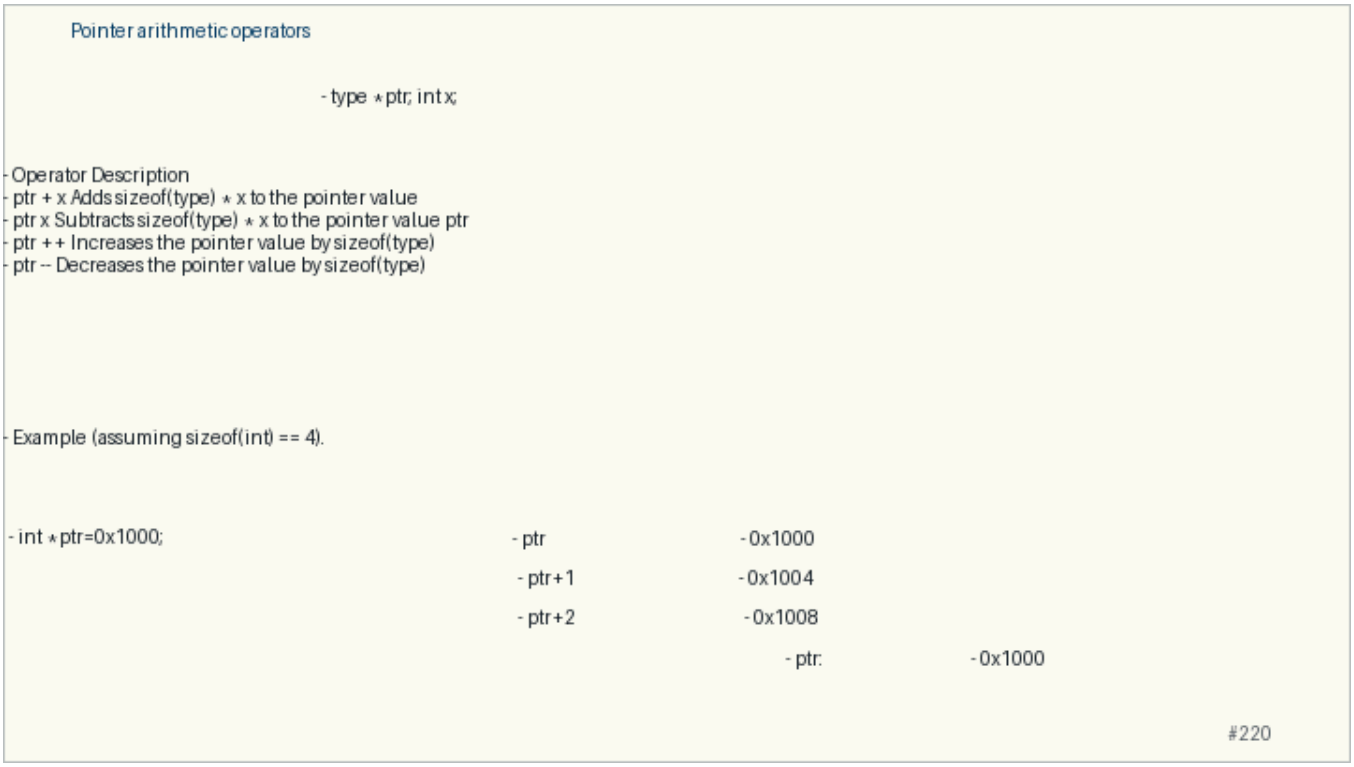
- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

4.5 Backup Index

Original slides 219-221



slide 219



slide 220

slide 221

Anchors: Pointer arithmetic; Pointer arithmetic operators; Pointer arithmetic: Problems!

4.6 Backup: Pointer arithmetic

Original slide 219

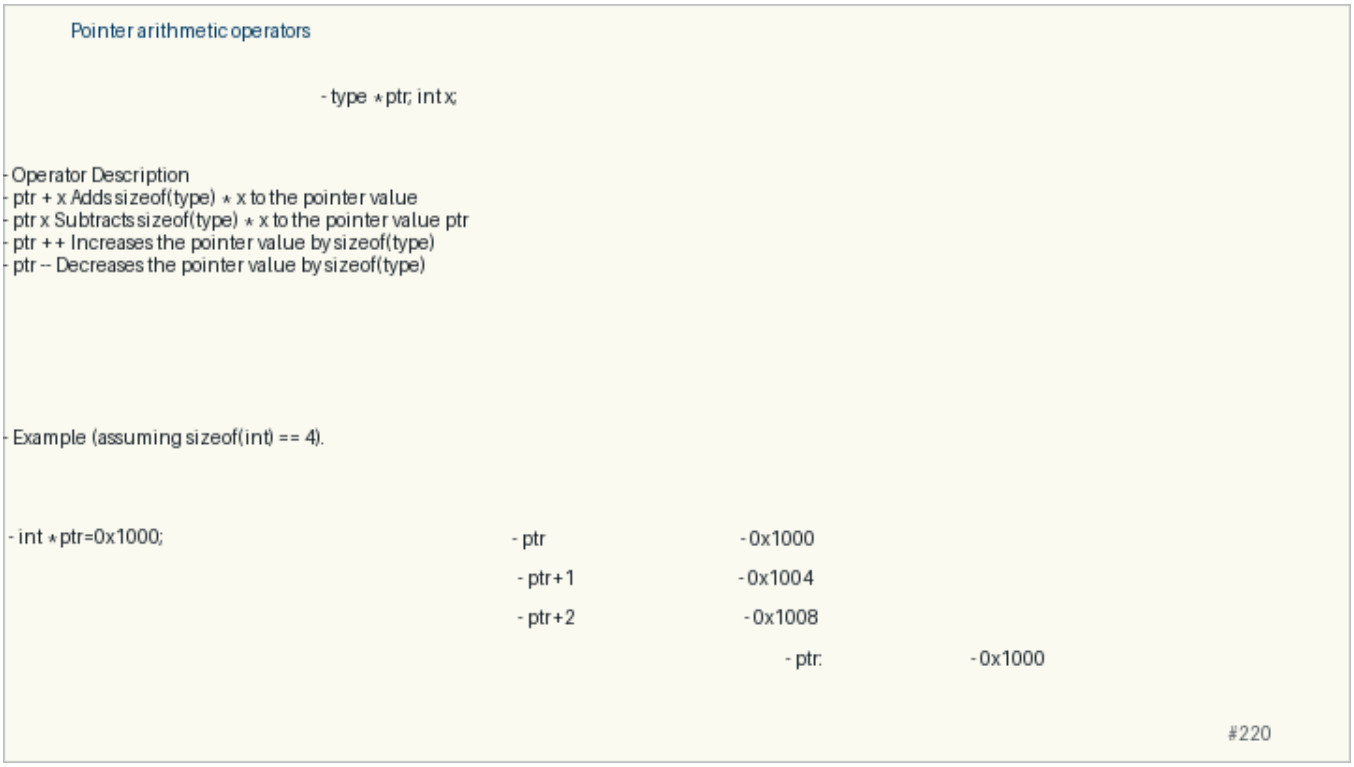
```
A pointer is NOT ONLY an address!
It provides information about
What type is the object stored in the address where the pointer points to
And thus, what is the size of the object.
```

```
Data type's size can be given by the sizeof() operator.
sizeof(int), sizeof(short)
The return value depends on the compiler and the target architecture!
```

4.7 Backup: Pointer arithmetic operators

Original slide 220

- type *ptr; int x;
- Operator Description
- ptr + x Adds sizeof(type) * x to the pointer value
- ptr x Subtracts sizeof(type) * x to the pointer value ptr
- ptr ++ Increases the pointer value by sizeof(type)
- ptr – Decreases the pointer value by sizeof(type)
- Example (assuming sizeof(int) == 4).
- int *ptr=0x1000;
- ptr
- 0x1000
- 0x1004
- ptr+1
- ptr+2
- 0x1008
- 0x1000
- ptr:



4.8 Backup: Pointer arithmetic: Problems!

Original slide 221

```
You can make a pointer point anywhere, the compiler will NOT complain. The following are valid:
ptr + 10000
ptr - 10000
char *ptr = (char *) 0x0000ffff
However, problems occur when you attempt to access an invalid memory address by dereferencing
the pointer
Usually, the operating system forbids access outside of the program's allocated memory
If the program can access the memory location, the data is not valid
If the program cannot access, it results to segmentation fault
The operating system sends the SEGFAULT signal to the process that caused the memory violation,
the process core dumps and terminates
```

- SECURITY ISSUES

Pointer arithmetic: Problems!

You can make a pointer point anywhere, the compiler will NOT complain. The following are valid:
ptr + 10000
ptr - 10000
char *ptr = (char *) 0x0000ffff
However, problems occur when you attempt to access an invalid memory address by dereferencing the
- SECURITY ISSUES

#221